MACHINE LEARNING LOGBOOK

Emmanuel Antwi Nyamaah

08/01/2026

A summary of activities from week 3 to week 11

Contents

WEEK 3 2

Task 1 2

Evaluation 4

WEEK 4 5

Task 1 5

Task 2 10

Evaluation 10

Week 5 11

Task 1 • *A subset of 2 features* 11

b- And compare the accuracy of your Random Forest Classifier. Which one is better? Discuss your observations and justify your answer. 14

What is the largest number of features you may have? 14

Evaluation 14

Week 6 15

Task 1 15

Evaluation 19

Week 7 19

Task 1 19

Task 2 23

Week 8 24

Task 1 24

Task 2 24

Week 9 27

Task 1 27

K = 5 28

K = 13 28

K = 15 29

Evaluation 29

Week 10 30

Task 1 30

Task 2 31

Kernal 3, 3 31

Kernal 5, 5 36

Evaluation 40

Week 11 41

Task 1 41
Task 2 41
Task 3 43
Task 4 44
Evaluation 44
References 45

WEEK 3

Task 1



Figure 1 - Importing libraries for clustering

Libraries for clustering and numpy were imported, and code to mount Google Drive was run to access files and define the foldername for data.



Figure 2 - loading cluster

In this visualisation, we loaded the data_kcluster.txt dataset into a numpy array named X. Then, we created an initial scatter plot of X to visualise the raw data points.

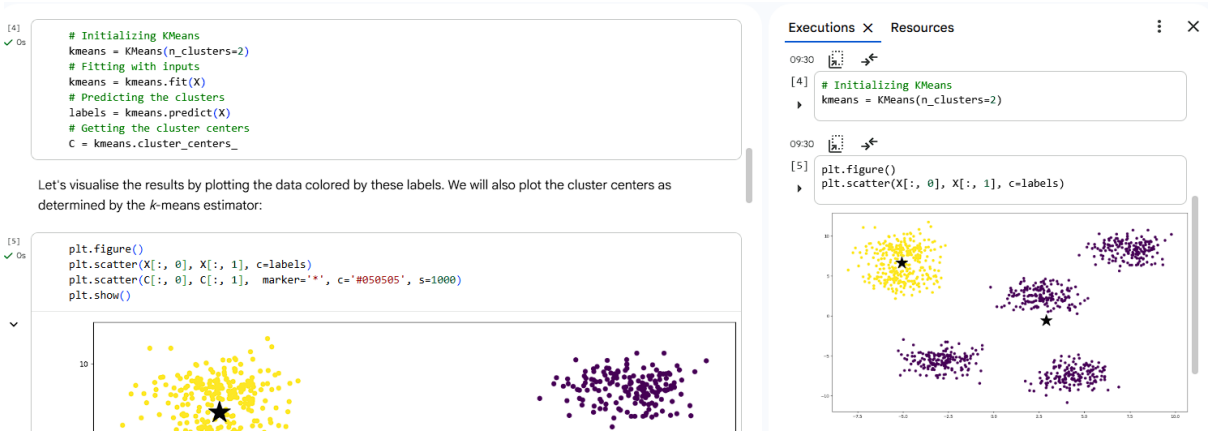


Figure 3 - fitting model

In this visualisation, I used K-Means to split my data into two clusters. Fitted the model and predicted labels for each point. Now I'm plotting the results with coloured and star-marked centres.

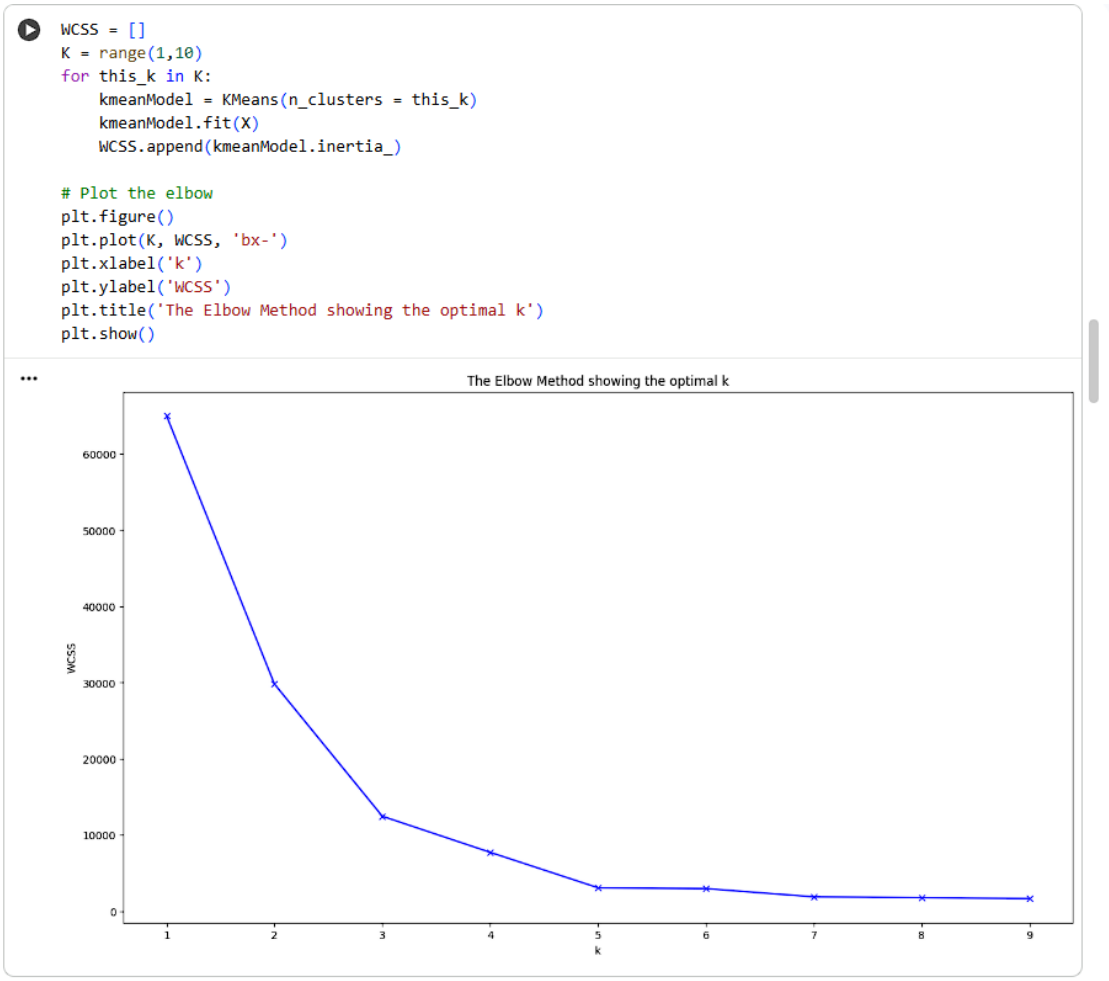


Figure 4 - graph for kMeans

I initialise an empty list, WCSS, and iterate over k values (1 to 9). For each k, I create a KMeans model, fit it to data X, and append its inertia_ (WCSS) to WCSS. This plots WCSS against k to find the elbow point and determine the optimal number of clusters. After identifying it, I run Kmeans with that k, fit the model, predict labels, and extract centres.

```
# Initializing KMeans
K_elbow = 4 # replace this value by what you found in elbow method
kmeans = KMeans(n_clusters = K_elbow)
# Fitting with inputs
kmeans = kmeans.fit(X)
# Predicting the clusters
labels = kmeans.predict(X)
# Getting the cluster centers
C = kmeans.cluster_centers_

plt.figure()
plt.scatter(X[:, 0], X[:, 1], c=labels)
#plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(C[:, 0], C[:, 1],  marker='*', c='#050505', s=1000)
plt.show()
```

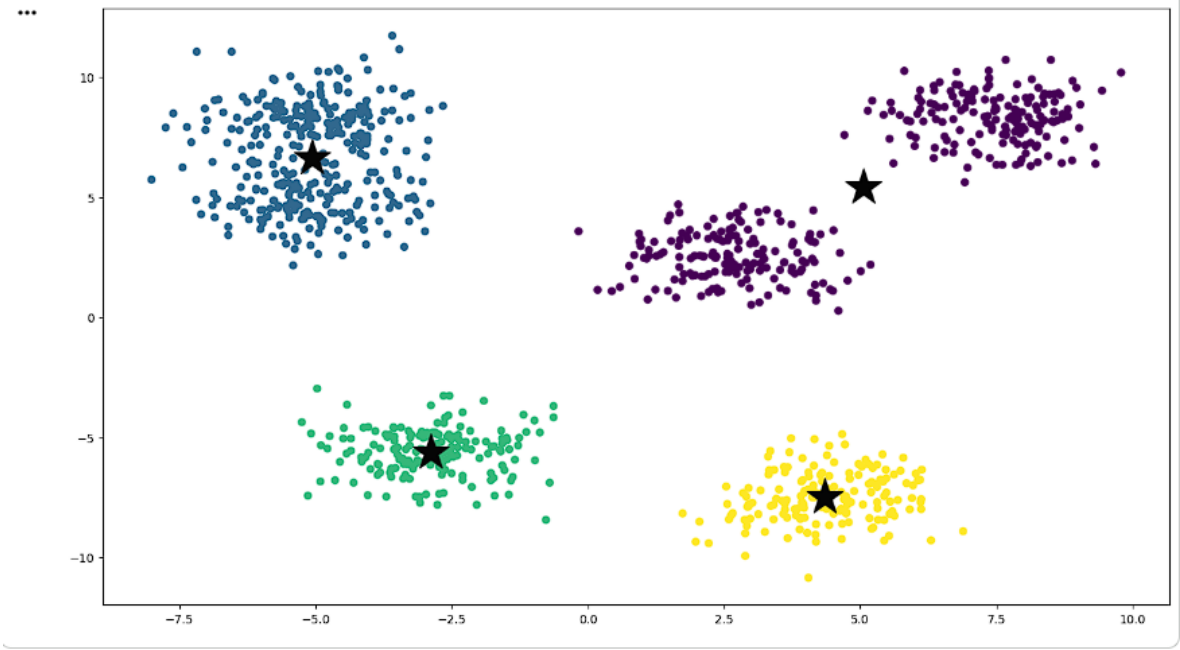


Figure 5 - Scatter plot

This visualisation creates a scatter plot of dataset X, colouring each data point according to its assigned. I ran K-Means with 4 clusters based on the elbow method. After fitting the model to my data, I predicted the cluster labels and plotted the results—each group in a different colour, with black stars marking the cluster centres.

Evaluation

I used the Elbow Method to estimate clusters but couldn't find a clear elbow point. I chose k=4 based on visual judgment, but the curve lacked a distinct bend, indicating

possible subtle or overlapping clustering. This shows the limitations of relying solely on WCSS and the need for other methods, such as silhouette scores or visual inspection.

WEEK 4

Task 1

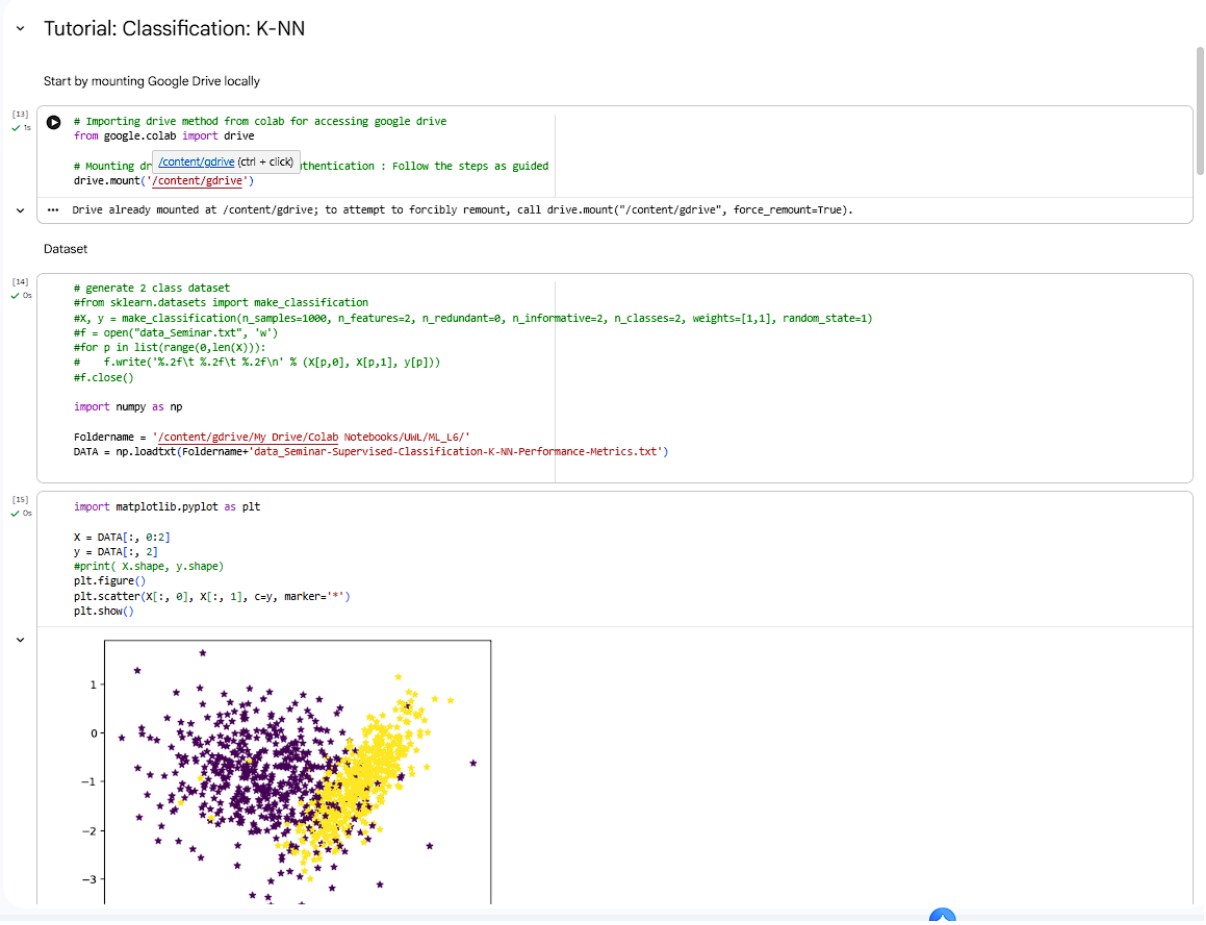


Figure 6 - KNN application

I mounted my Google Drive, generated the KNN performance metrics dataset and loaded labelled data for training. From the scatter plot, I could see that there are only two classes distributed before applying K-NN



Figure 7 - training K-NN

In the visualisation, I first set n_neighbours to 7 to get the results. The dataset was split into training and testing sets using an 80/20 ratio. Then I trained a K-NN classifier with 7 neighbours on the training data so I can later evaluate how well it predicts the test set.

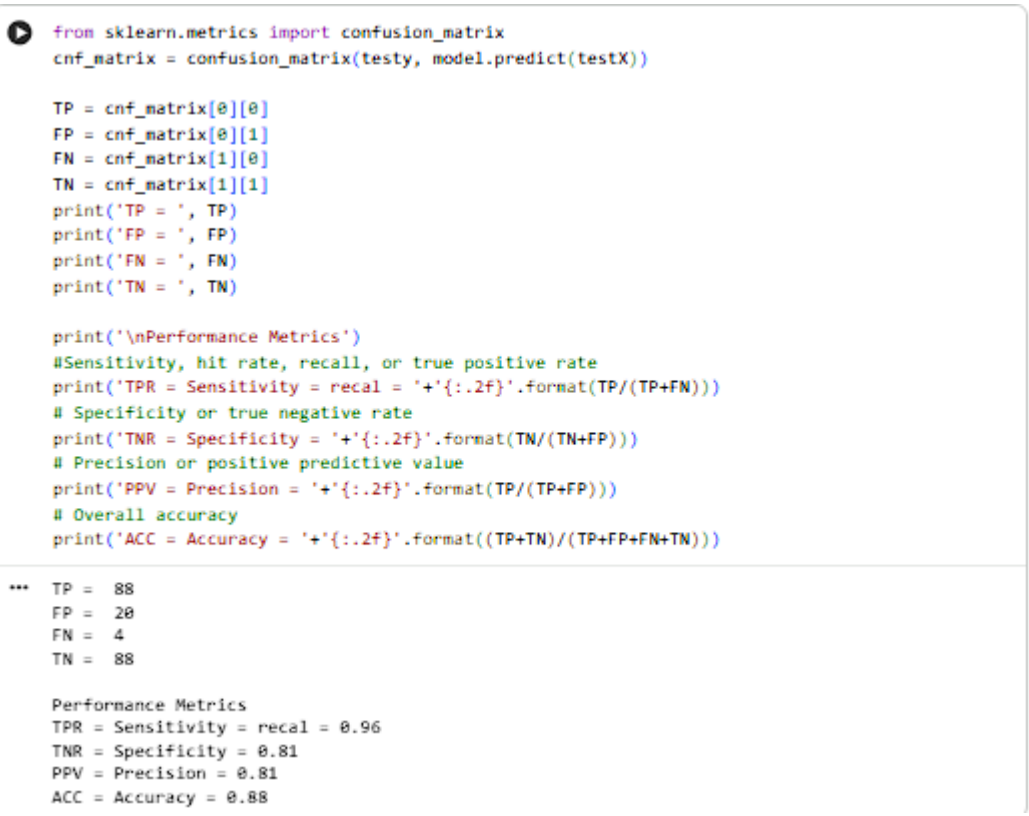


Figure 8 - K-NN sensitivity

The k=7 K-NN model achieved 96% sensitivity, 81% specificity, 88% accuracy, and an F1 score of 0.88, confirming reliable performance across classes. It identifies true positives well but could improve false positive handling.

```
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
#    if normalize:
#        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
#        print("Normalized confusion matrix")
#    else:
#        print('Confusion matrix')

    #print(cm)

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    import itertools
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.tight_layout()
```

Figure 9 - confusion matrix

A custom function plotted the confusion matrix with colour shading and text, making classifier performance easier to interpret. It supports raw and normalised values, complementing numeric metrics and highlighting misclassifications or class imbalance.

```
class_names=['Negative', 'Positive']
plt.figure()
plot_confusion_matrix(cnf_matrix, classes=class_names, title='Confusion matrix')

## Plot normalized confusion matrix
#plt.figure()
#plot_confusion_matrix(cnf_matrix, classes=class_names, normalize=True, title='Normalized confusion matrix')
```

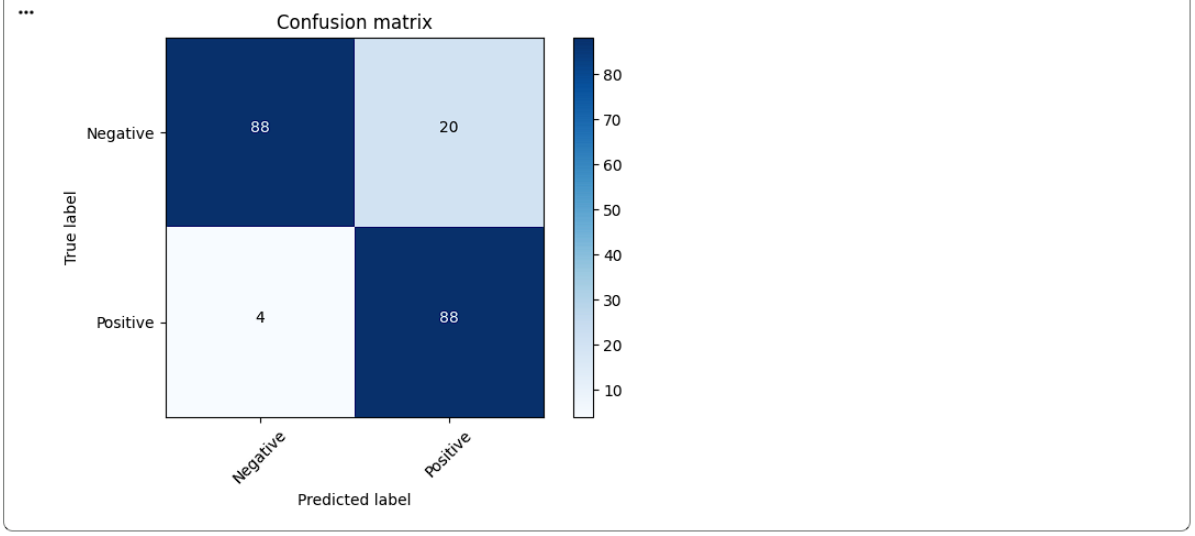


Figure 10 - predicted label from the confusion matrix

The custom confusion matrix clearly compared actual labels with predictions. Strong diagonal values confirmed good classification, while misclassifications were easy to spot, reinforcing earlier performance metrics.

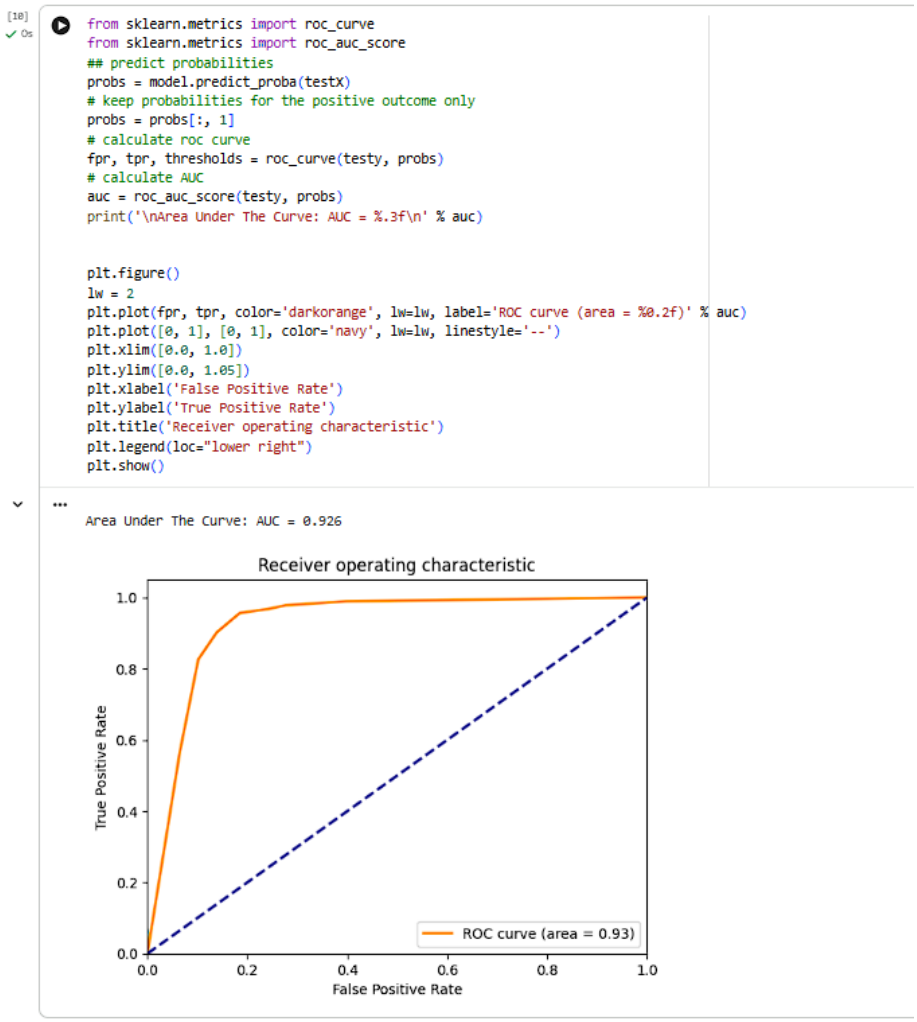


Figure 11 - receiver operating characteristic

I completed my K-NN classification by plotting the ROC curve and calculating the AUC score. The curve showed clear class separation, and with an AUC of 0.93, the model effectively distinguishes positive and negative cases. This confirms earlier metrics and boosts confidence in the classifier's performance.

```
09:59
[17]
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)

KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)

09:59
[18]
from sklearn.metrics import confusion_matrix
cnf_matrix = confusion_matrix(testy, model.predict(testX))

TP =  86
FP =  22
FN =  6
TN =  86

Performance Metrics
TPR = Sensitivity = recal = 0.93
TNR = Specificity = 0.80
PPV = Precision = 0.80
ACC = Accuracy = 0.86

09:59
[19]
# print('\nPerformance Metrics using sklearn')
# from sklearn.metrics import accuracy_score

f1_score = 0.86

09:59
[20]
def plot_confusion_matrix(cm, classes,
                          normalize=False,

09:59
[21]
class_names=['Negative', 'Positive']
plt.figure()
```

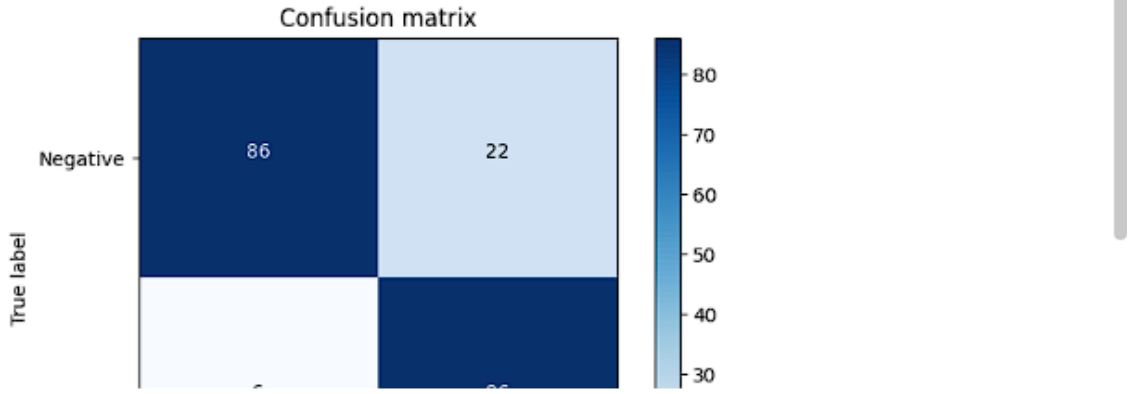


Figure 12 - k=3 training

After retraining with k=3, accuracy was 86%, sensitivity 93%, and F1 score 0.86. Despite balanced performance, misclassifications increased compared to k=7, showing that fewer neighbours may not be optimal and highlighting how k impacts recall and precision.

```
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
## predict probabilities
probs = model.predict_proba(testX)
# keep probabilities for the positive outcome only
probs = probs[:, 1]
# calculate roc curve
fpr, tpr, thresholds = roc_curve(testy, probs)
# calculate AUC
auc = roc_auc_score(testy, probs)
print('\nArea Under The Curve: AUC = %.3f\n' % auc)


plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange', lw=lw, label='ROC curve (area = %0.2f)' % auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```

```
Area Under The Curve: AUC = 0.904
```

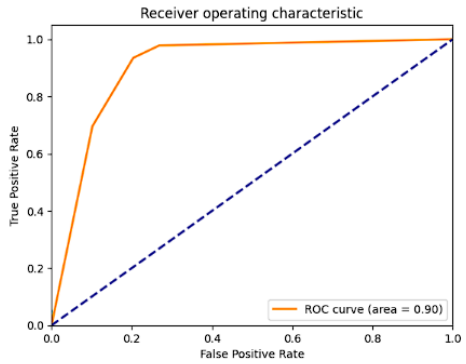


Figure 13 - ROC curve

The ROC curve (AUC = 0.90) confirmed strong K-NN performance. With k=3, accuracy and F1 dropped to 86% and misclassifications rose, showing k=7 provides a better balance of recall and precision.

Task 2

The F1 score shows how well a model balances precision and recall, especially with class imbalance or equal importance of errors. I compared two K-NN models: k=7 had an F1 of 0.88, while k=3 scored 0.86. Both performed well, but k=7 better balanced recall and precision, confirming it's the better choice for this dataset.

Evaluation

The k=7 K-NN model achieved 96% sensitivity, 81% specificity, 88% accuracy, and an F1 of 0.88, outperforming k=3 (F1 = 0.86). The ROC curve (AUC = 0.90) confirmed k=7 offers a better balance of precision and recall.

Week 5

Task 1

- *A subset of 2 features*

```
# Load scikit's random forest classifier library
from sklearn.ensemble import RandomForestClassifier

# Create a random forest Classifier. By convention, clf means 'Classifier'
# We can specify the number of features using max_features
clf = RandomForestClassifier(max_features=2, random_state=0)

# Train the Classifier to take the training features and learn how they relate
# to the training y (the species)
clf.fit(train[features], y)
```

```
RandomForestClassifier(max_features=2, random_state=0)
```

Figure 14 - Inserting feature

This image shows the implementation of the random forest classifier with max 2 feature

```
# View a list of the features and their importance scores
#list(zip(train[features], clf.feature_importances_))
feature_imp = pd.Series(clf.feature_importances_,index=iris.feature_names).sort_values(ascending=False)
feature_imp
```

	0
petal length (cm)	0.446485
petal width (cm)	0.446313
sepal length (cm)	0.084740
sepal width (cm)	0.022461

dtype: float64

Figure 15 - features

The image illustrates a random forest classifier using 2 features, showing the model selecting only two features per tree split. It also displays the output confirming these parameters.

```
y_pred = clf.predict(test[features])
y_pred

y_test = pd.factorize(test['species'])[0]
y_test

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

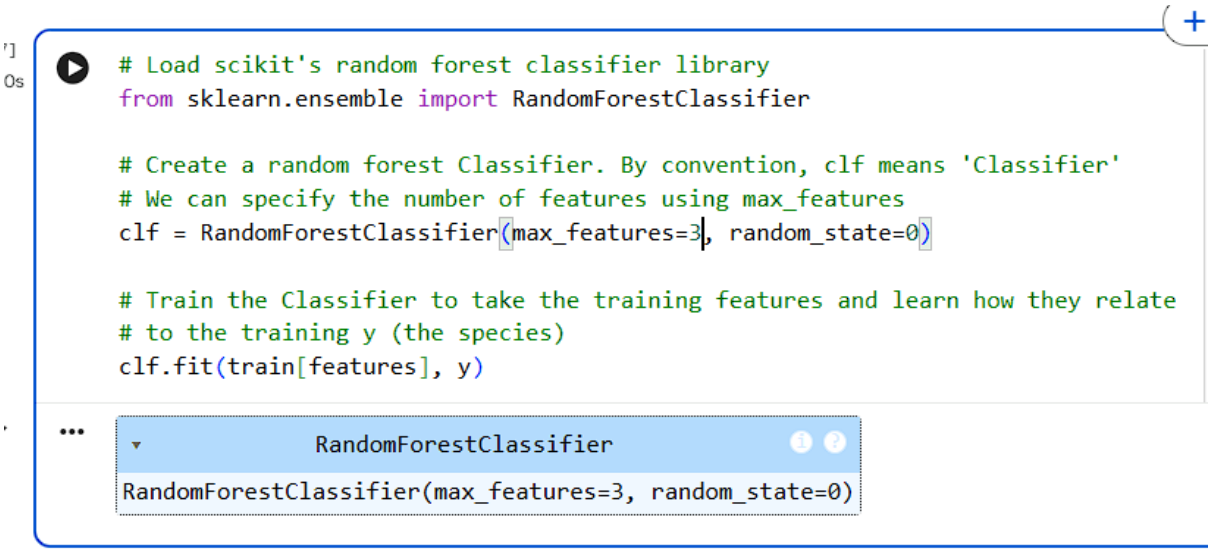
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.9375
```

Figure 16accuracy for 2 features

This image shows that feature 2 achieves impressive accuracy of 0.9375

A subset of 3 features



```
# Load scikit's random forest classifier library
from sklearn.ensemble import RandomForestClassifier

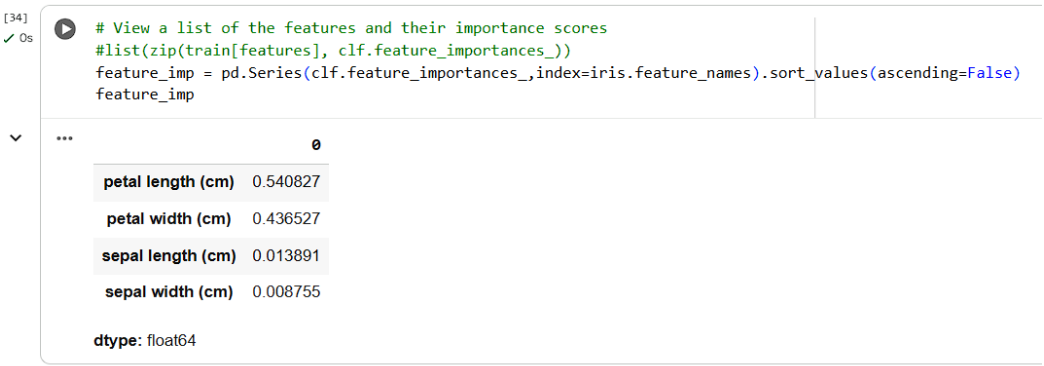
# Create a random forest Classifier. By convention, clf means 'Classifier'
# We can specify the number of features using max_features
clf = RandomForestClassifier(max_features=3, random_state=0)

# Train the Classifier to take the training features and learn how they relate
# to the training y (the species)
clf.fit(train[features], y)
```

```
RandomForestClassifier(max_features=3, random_state=0)
```

Figure 17 - max feature 3

This image shows the implementation of the random forest classifier with max 3 feature



```
# View a list of the features and their importance scores
#list(zip(train[features], clf.feature_importances_))
feature_imp = pd.Series(clf.feature_importances_,index=iris.feature_names).sort_values(ascending=False)
feature_imp
```

	0
petal length (cm)	0.540827
petal width (cm)	0.436527
sepal length (cm)	0.013891
sepal width (cm)	0.008755

dtype: float64

Figure 18 - feature 3

The image displays the implementation of a random forest classifier with three feature importance scores, highlighting petal length and petal width (both 0.446) as the most important features for classification, while sepal length and width are less significant.

```
y_pred = clf.predict(test[features])
y_pred

y_test = pd.factorize(test['species'])[0]
y_test

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.96875
```

Figure 19 - accuracy for feature 3

This image shows features 3 achieving impressive accuracy of 0.96875. The highest.

- *A subset of 4 features*

```
# Load scikit's random forest classifier library
from sklearn.ensemble import RandomForestClassifier

# Create a random forest Classifier. By convention, clf means 'Classifier'
# We can specify the number of features using max_features
clf = RandomForestClassifier(max_features=4, random_state=0)

# Train the Classifier to take the training features and learn how they relate
# to the training y (the species)
clf.fit(train[features], y)
```

```
RandomForestClassifier(max_features=4, random_state=0)
```

Figure 20 - feature 4

This image shows the implementation of the random forest classifier with max 3 feature

```
# View a list of the features and their importance scores
#list(zip(train[features], clf.feature_importances_))
feature_imp = pd.Series(clf.feature_importances_,index=iris.feature_names).sort_values(ascending=False)
feature_imp
```

	0
petal length (cm)	0.535099
petal width (cm)	0.448016
sepal length (cm)	0.009444
sepal width (cm)	0.007442

dtype: float64

Figure 21 - feature 4

This image shows the feature importance scores from the Random Forest Classifier, highlighting that petal length (0.535099) and petal width (0.448016) are the most significant predictors for classifying the Iris dataset, whereas the sepal measurements have minimal impact.

```
y_pred = clf.predict(test[features])
y_pred

y_test = pd.factorize(test['species'])[0]
y_test

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.9375
```

Figure 22 - accuracy for feature 4

This image shows features 4 achieving accuracy of 0.9375, same as feature 2.

b- And compare the accuracy of your Random Forest Classifier. Which one is better? Discuss your observations and justify your answer.

The 3-feature Random Forest achieved 0.96875 accuracy, outperforming the 4-feature model (0.9375). Petal length and width are the most influential features; limiting the number of features reduces correlation, increases diversity, and improves generalisation over all features.

What is the largest number of features you may have?

The maxfeatures maximum is the total number of features, 4 for Iris. Setting it to this value considers all features at each split, making the Random Forest a Bagged Ensemble rather than a true Random Forest.

Evaluation

You tested three Random Forest configurations on the Iris dataset. The max_features=3 model (Accuracy: 0.96875) outperformed the max_features=2 and max_features=4 models (Accuracy: 0.9375), confirming that tuning max_features is important. Smaller feature subsets can help Random Forest outperform standard Bagged Ensembles (max_features=4).

Week 6

Task 1



Figure 23 - importing perceptron file

I successfully imported data_Perceptron.txt file into a pandas DataFrame named df_perceptron. The first 5 rows are shown in the image above.

```
df_perceptron = pd.read_csv('/content/data_Perceptron.txt', sep='\t', header=None)
# Define the activation function  that returns either 1 or 0
def activation(x):
    return 1 if x >= 1 else 0

# A function to calculate the unit vector of our weights vector
def calc_unit_vector(x):
    return x.transpose() / np.sqrt(x.transpose().dot(x))

# A function that returns values that lay on the hyperplane
def calc_hyperplane(X, w):
    return np.ravel([-(w[0] + x * w[1]) / w[2] for x in X])
```

Figure 24 - perceptron binary classifier

This code sets up a perceptron-based binary classifier by loading data, defining an activation function, normalising weights, and computing the decision boundary for visualisation.

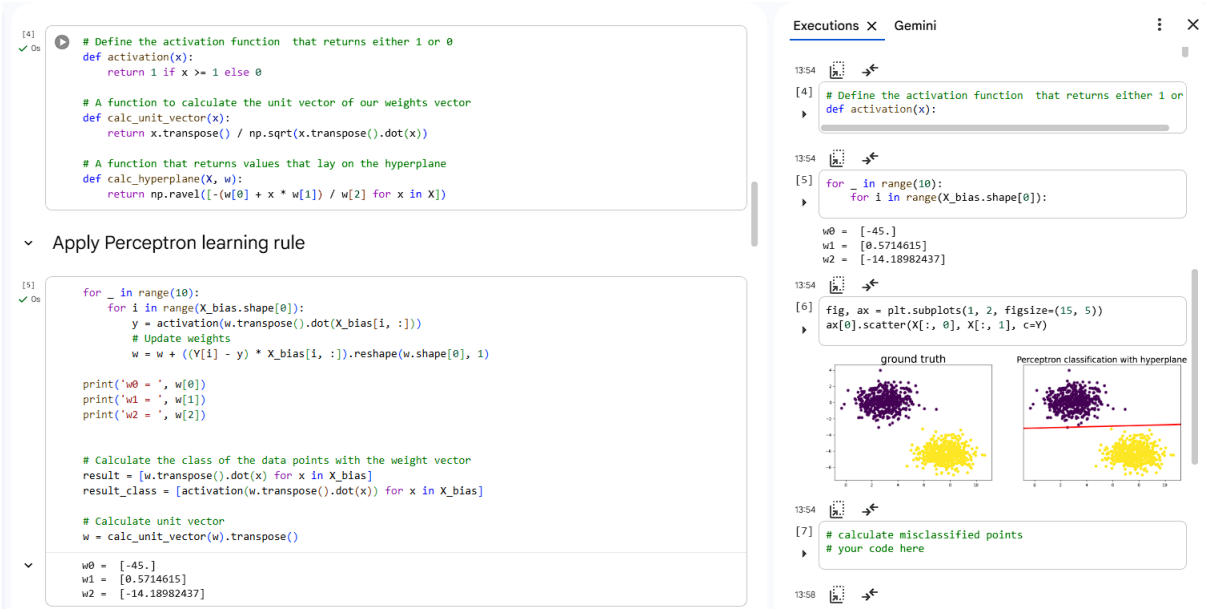


Figure 25 - misclassified points

Misclassified points were identified by comparing ground truth labels (Y) with Perceptron results (result_class) to count errors and assess linear separability.

A scatter plot then visualised these misclassified points, aiding understanding of the misclassification pattern.

```
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
ax[0].scatter(X[:, 0], X[:, 1], c=Y)
ax[0].set_title('ground truth', fontsize=25)

ax[1].scatter(X[:, 0], X[:, 1], c=result_class)
ax[1].plot([-20, 20], calc_hyperplane([-20, 20], w), lw=3, c='red')
ax[1].set_xlim(ax[0].get_xlim())
ax[1].set_ylim(ax[0].get_ylim())
ax[1].set_yticks([])
ax[1].set_title('Perceptron classification with hyperplane', fontsize=20)
plt.show()
```

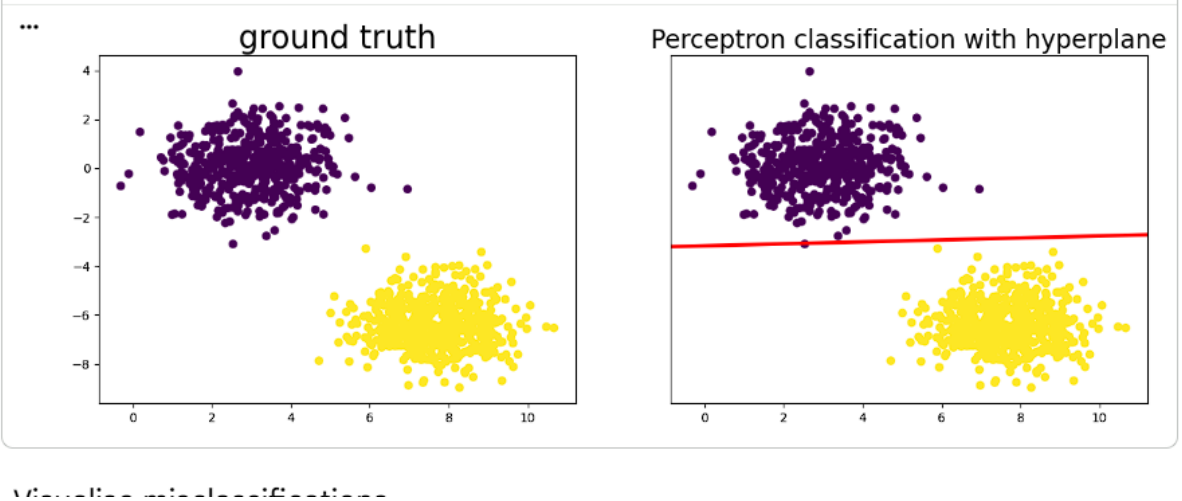


Figure 26 - hyperplane

The second plot shows Perceptron classification with a hyperplane, illustrating how the algorithm separates data points and the decision boundary it learned. This helps understand its performance in distinguishing the two groups.

```
y_pred = np.array([activation(w.T.dot(X_bias[i, :])) for i in range(X_bias.shape[0])])

# Identify misclassified points (1 if misclassified, 0 if correct)
misclassified = (y_pred.flatten() != Y).astype(int)

# Compute percentage of misclassified points
misclassified_percentage = (misclassified.sum() / len(Y)) * 100
print(f"Percentage of misclassified points: {misclassified_percentage:.2f}%")

# Plot misclassified points
fig, ax = plt.subplots(1, 1, figsize=(5, 6))
ax.scatter(X[:, 0], X[:, 1], c=misclassified, cmap='coolwarm')
ax.set_title('Misclassified points', fontsize=20)
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
```

```
Percentage of misclassified points: 20.60%
```

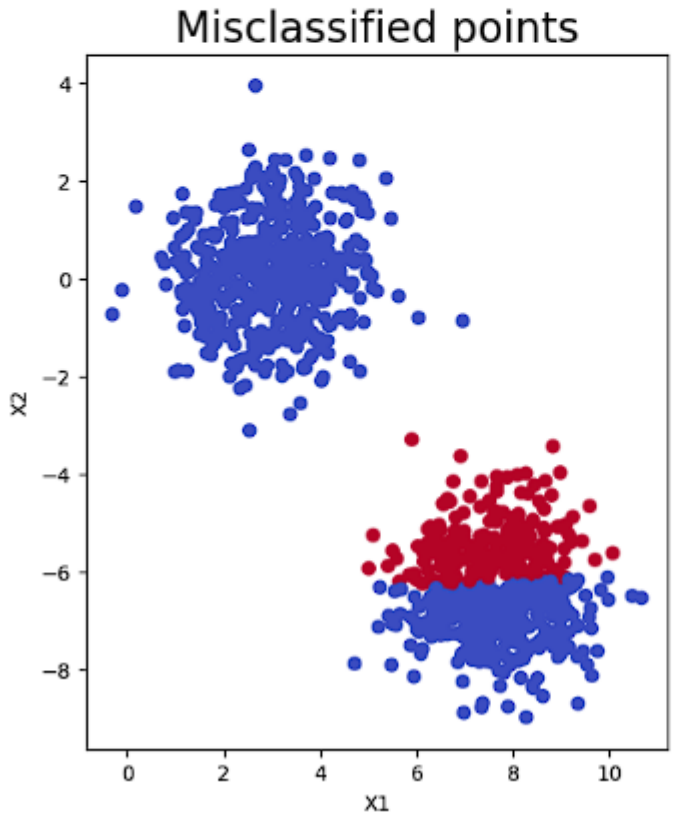


Figure 27 - misclassified points graph

This visualisation compares predictions to accurate labels, identifies incorrect ones, and calculates a 20.60% misclassification rate. It uses a scatter plot with points coloured by correctness, helping spot error patterns.

The data isn't linearly separable, as shown by overlapping purple and yellow points and the perceptron's hyperplane misclassifying several points. The misclassification rate of 20.60% is derived from comparing predictions to accurate labels, highlighting that a linear boundary isn't sufficient.

Evaluation

The Perceptron learning rule is a simple algorithm for binary classification that finds a hyperplane to separate data into two categories. It starts with an initial guess, iteratively examines data points, and adjusts the separating line when misclassifications occur. This repeats until all points are correctly separated, assuming the data is linearly separable.

Week 7

Task 1

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn import metrics

data = np.loadtxt('data_SVM.txt')
X = data[:, 0:2]  # Features
y = data[:, 2]
# use seaborn plotting defaults
import seaborn as sns; sns.set()
```

Figure 28 - importing datasets

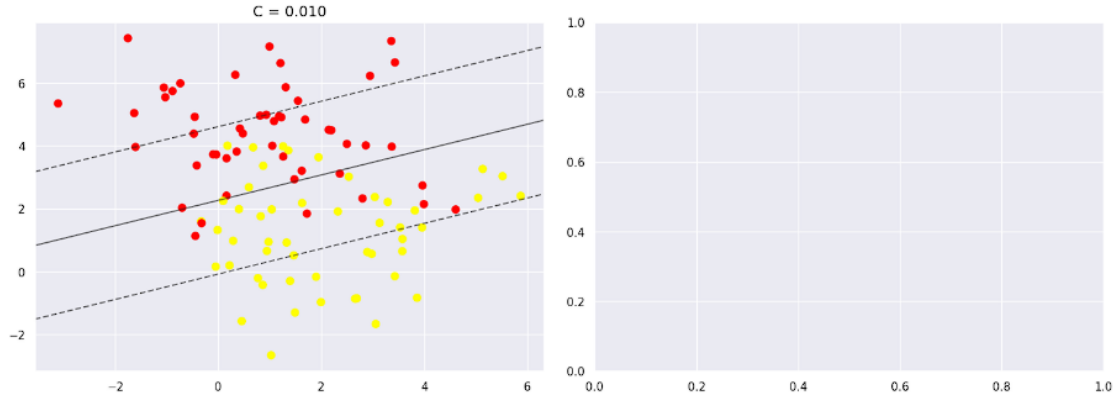
I started off by importing the data into the script to load up the dataset for SVM, identifying its features.

```
[62]
✓ 0s
C = 0.01 # C parameter
model = SVC(kernel='linear', C=C).fit(X, y)
y_pred = model.predict(X)
```

```
[63]
✓ 0s
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)

ax[0].scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model, ax[0])
ax[0].set_title('C = {0:.3f}'.format(C), size=14)
```

```
Text(0.5, 1.0, 'C = 0.010')
```




```
[64]
✓ 0s
misclassified = (y != y_pred)

total_misclassified = np.sum(misclassified)
percentage_misclassified = (total_misclassified / len(y)) * 100

print('Total number of misclassified points = ', total_misclassified)
print('Percentage of misclassified points = ', percentage_misclassified)
```

```
Total number of misclassified points =  15
Percentage of misclassified points =  15.0
```

```
[65]
✓ 0s
ax[1].scatter(X[:, 0], X[:, 1], c = misclassified, cmap='autumn')
plot_svc_decision_function(model, ax[1])
ax[1].set_title('misclassified points', fontsize=20)
```

```
Text(0.5, 1.0, 'misclassified points')
```

Figure 29 - C=0.01 training

With C = 0.01, the model trained on data_SVM.txt, which has two features and labels, misclassified 15 points, a 15% error rate. This high error indicates a low C encourages a wider margin, allowing more misclassifications to reduce overfitting, especially with noisy data. The decision boundary plot shows this softer margin influenced by the regularisation parameter.

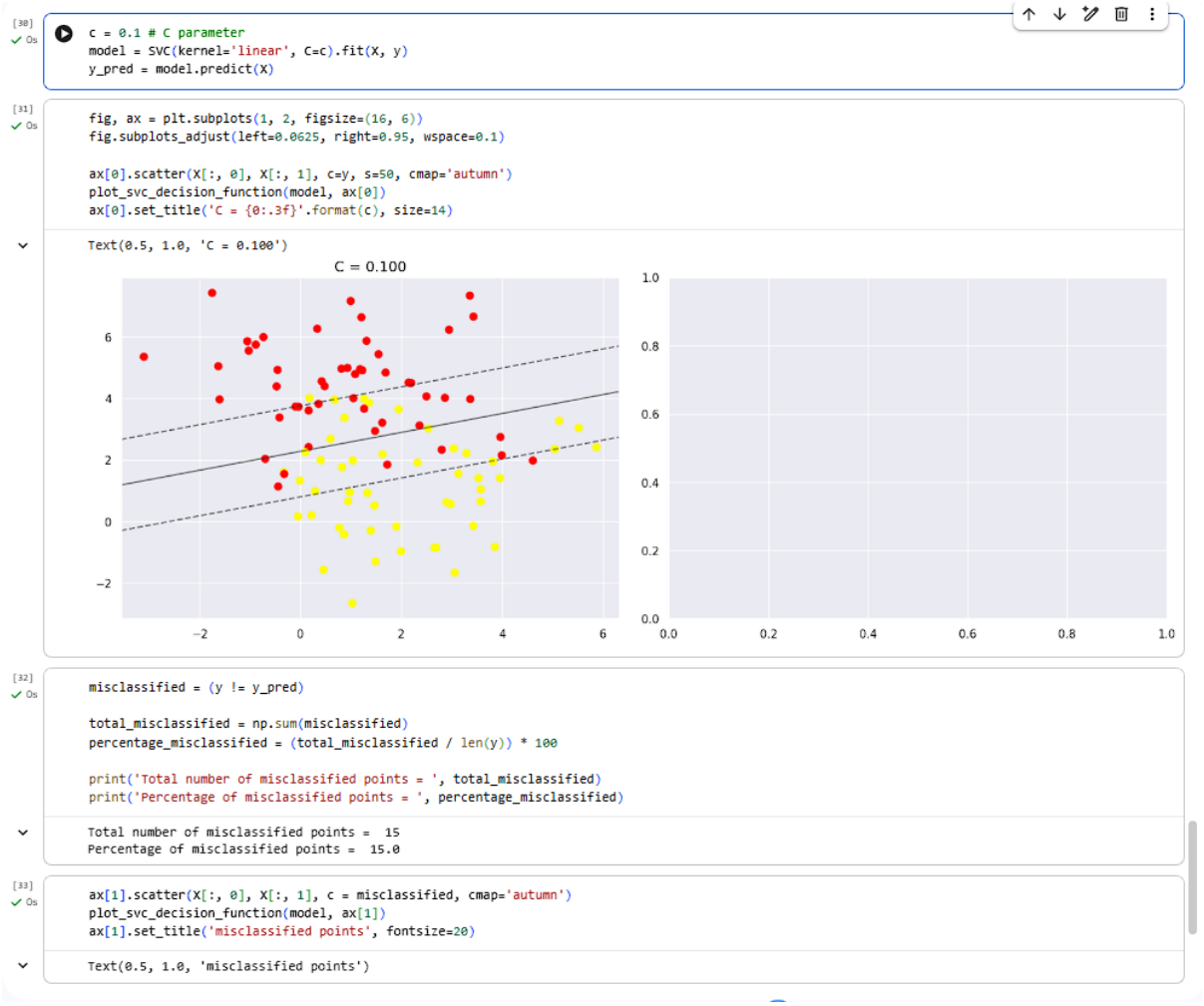


Figure 30 - C=0.1 training

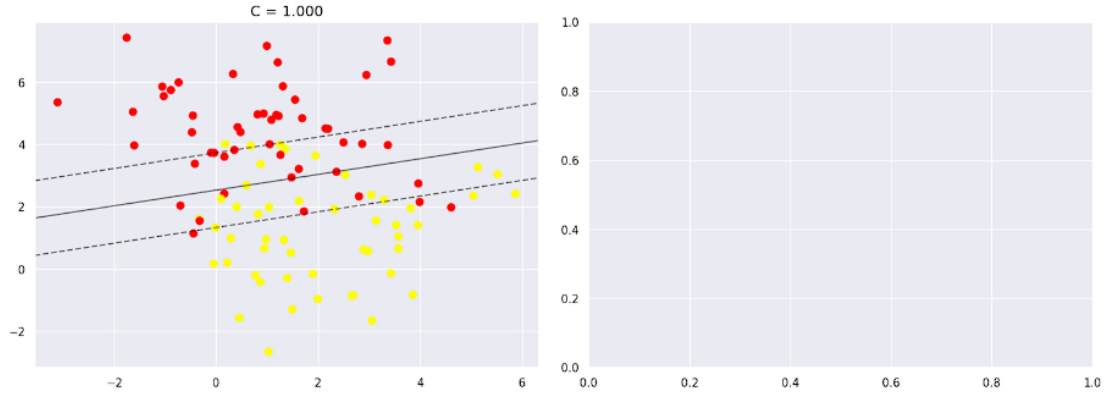
With C = 0.1, the model misclassified 15 points, a 15% error rate, same as C = 0.01. Slight C increase didn't significantly affect the model, and the decision boundary stayed largely the same, still allowing some misclassifications.

```
c = 1.0 # C parameter
model = SVC(kernel='linear', C=c).fit(X, y)
y_pred = model.predict(X)
```

```
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)

ax[0].scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model, ax[0])
ax[0].set_title('C = {0:.3f}'.format(c), size=14)
```

```
Text(0.5, 1.0, 'C = 1.000')
```




```
misclassified = (y != y_pred)

total_misclassified = np.sum(misclassified)
percentage_misclassified = (total_misclassified / len(y)) * 100

print('Total number of misclassified points = ', total_misclassified)
print('Percentage of misclassified points = ', percentage_misclassified)
```

```
Total number of misclassified points =  17
Percentage of misclassified points =  17.0
```

```
ax[1].scatter(X[:, 0], X[:, 1], c = misclassified, cmap='autumn')
plot_svc_decision_function(model, ax[1])
ax[1].set_title('misclassified points', fontsize=20)
```

```
Text(0.5, 1.0, 'misclassified points')
```

Figure 31 - C= 1 training

With C = 1.0, the model misclassified 17 points (17%), which is slightly worse than lower C values. A higher C value narrows the margin but diminishes generalisation capabilities, thereby increasing the risk of overfitting on noisy data. This underscores the importance of carefully tuning C to achieve optimal performance.

Compare your SVM results with those from the Perceptron in week-6.

The Perceptron had a 20.60% error rate because the dataset wasn't linearly separable. It couldn't correctly classify overlapping points, showing its limitation with noisy or non-linear data.

The SVM classifier outperformed the Perceptron across all regularisation values: C 0.01 and 0.1 led to 15% misclassification, while C 1.0 resulted in 17%. Even with C = 0.01, SVM achieved lower errors by allowing flexibility and improving generalisation. Its ability to maximise margins and use support vectors made it more robust, especially with overlapping classes, demonstrating SVM's advantage over the Perceptron on non-separable data.

Task 2

Week 8

Task 1

A single-layer perceptron can only solve linearly separable problems due to straight decision boundaries. A multi-layer perceptron (MLP), with hidden layers and non-linear activation functions, learns complex patterns, captures hierarchical features, and can approximate any continuous function (Universal Approximation Theorem). This makes MLPS more powerful for tasks from pattern recognition to natural language processing.

Task 2

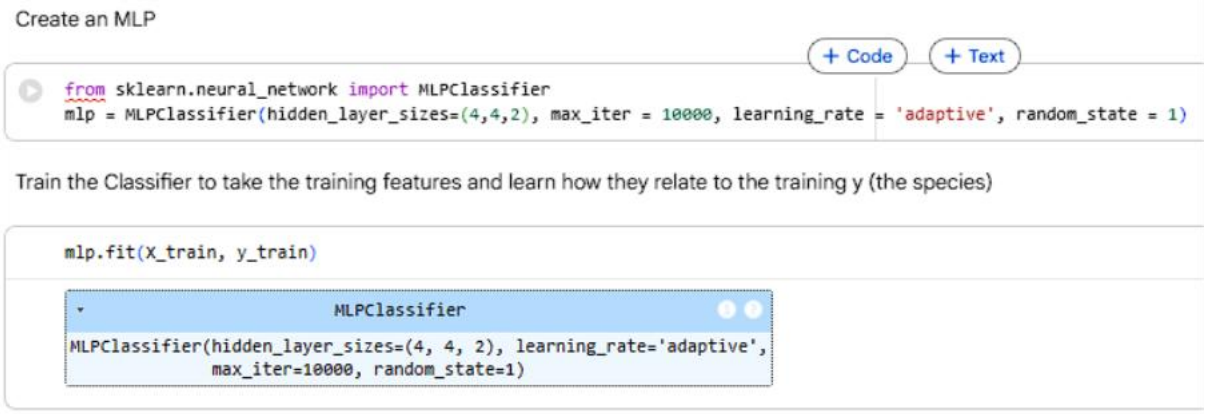


Figure 32 - hidden layers

In this visualisation, I just ran the original data given, and I came across a network with 3 hidden layers: first layer with 4 neurons, second layer with 4 neurons, and third layer with 2 neurons.

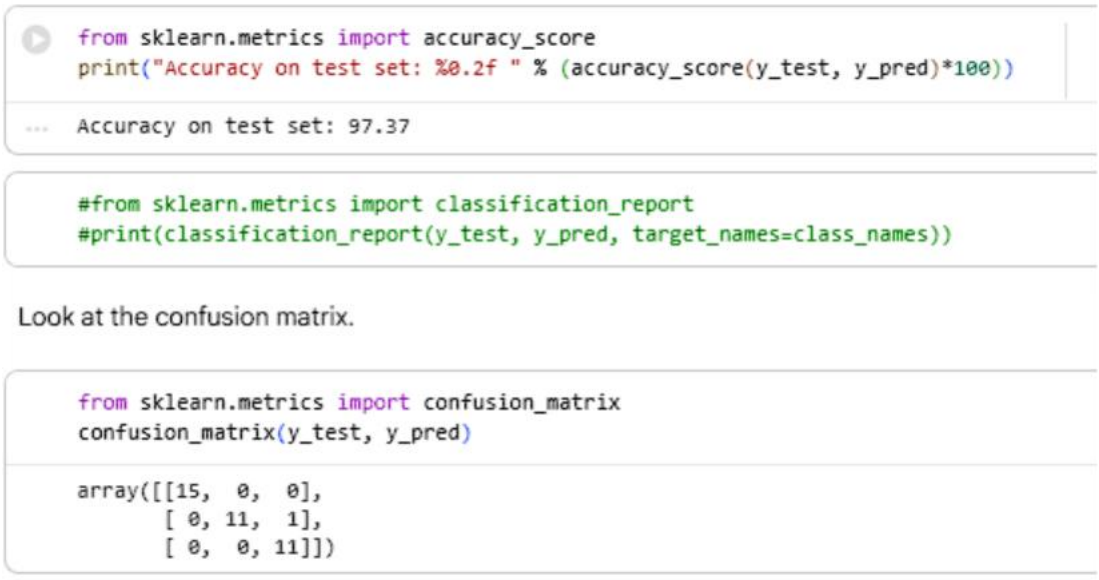


Figure 33 - confusion matrix

After running the code, I achieved a test set accuracy of 97.37%. Using a confusion matrix, I identified errors; most were correctly classified, but one was misclassified. My MLP model evaluated with cross-validation achieved 97% ± 6% accuracy, and changing architecture to (10,5) resulted in perfect accuracy.

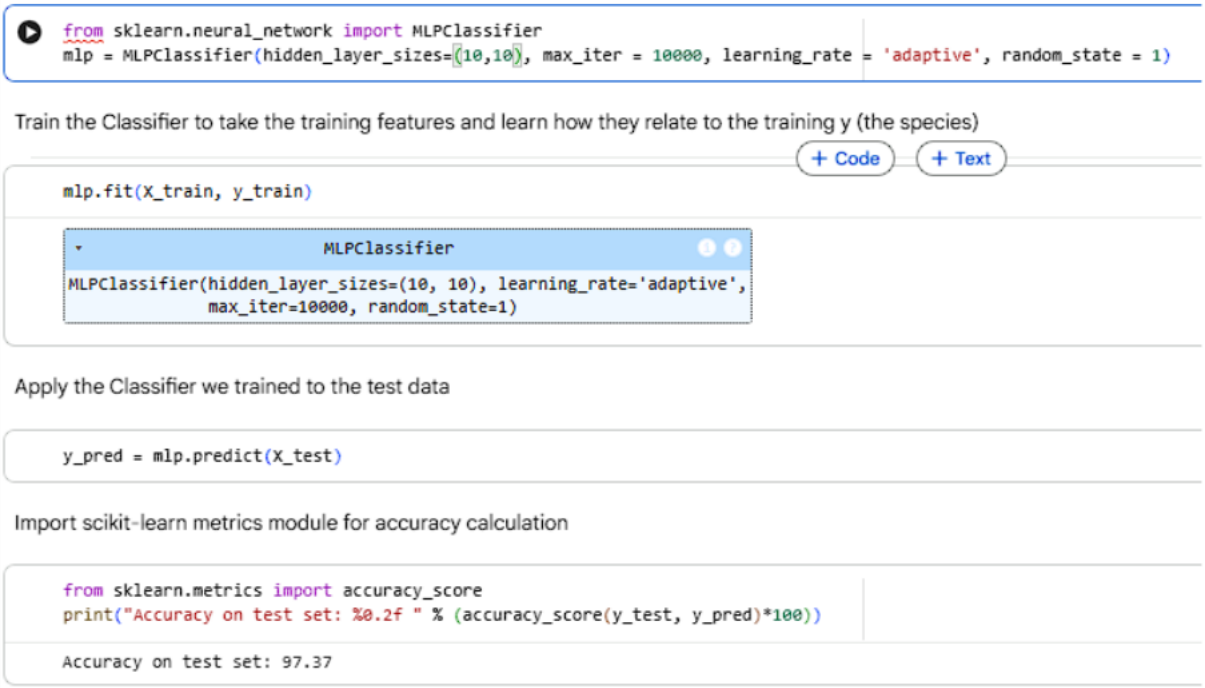


Figure 34 - accuracy for confusion matrix

Using a (10,10) architecture, I tested an MLP classifier and achieved 97.37% accuracy. The confusion matrix showed one misclassification, indicating the model was nearly perfect but not 100%. This experiment improved my understanding of how neuron count affects performance.

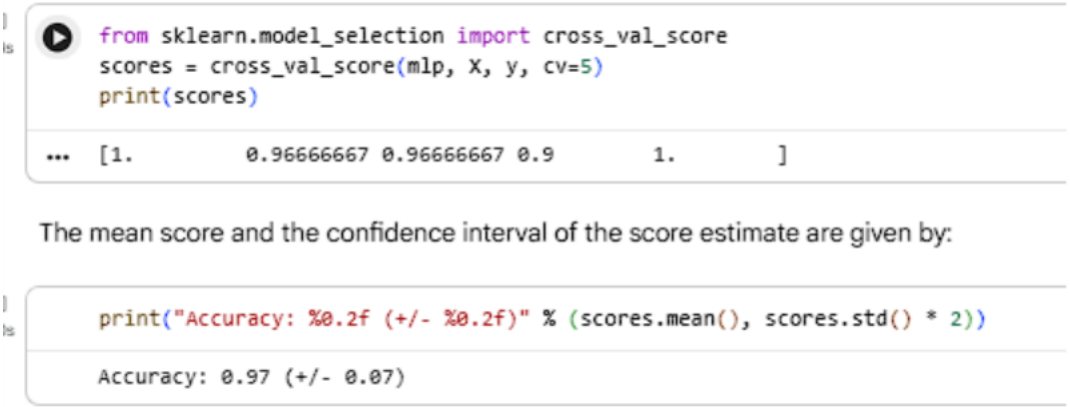


Figure 35 - accuracy for matrix

My MLP model achieved 90-100% scores after 5-fold cross-validation, with a mean accuracy of 97% ± 7%, showing good generalisation and consistent performance. The confusion matrix confirmed reliability, with only one misclassification.

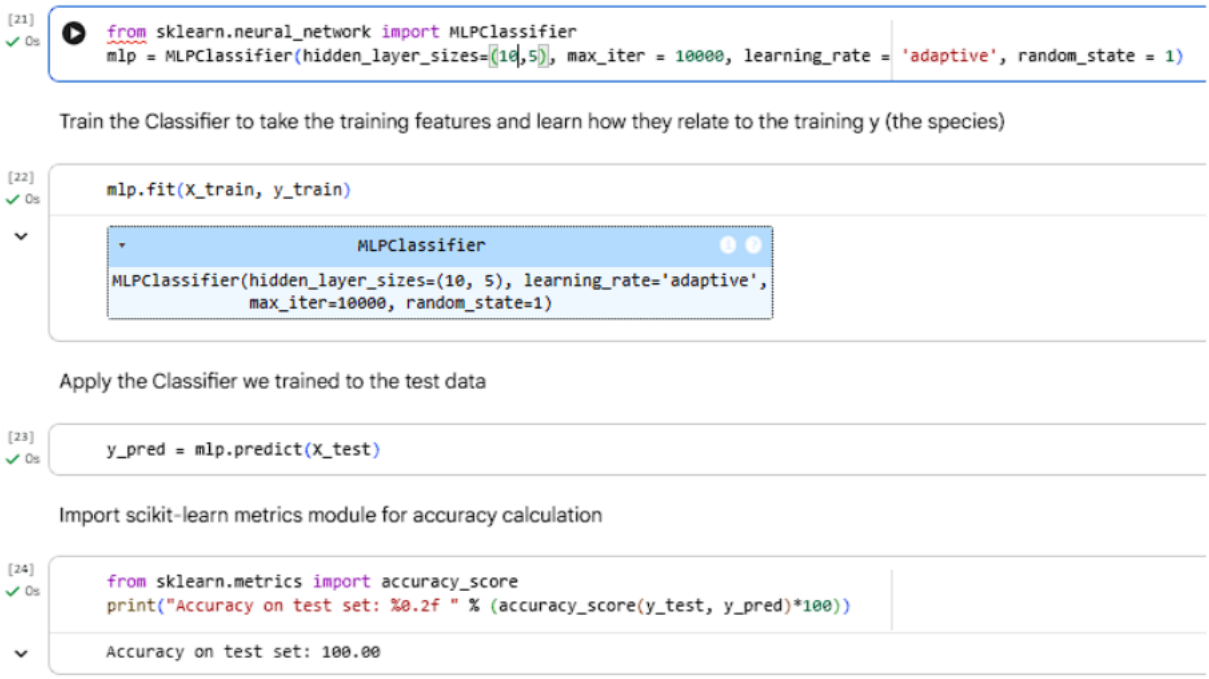


Figure 36 - (10,5) architecture

Using a (10, 5) architecture, I trained an MLP classifier achieving 100% accuracy on the test set, with a diagonal confusion matrix showing all samples predicted correctly. This indicates that all Iris dataset patterns were captured without overfitting.

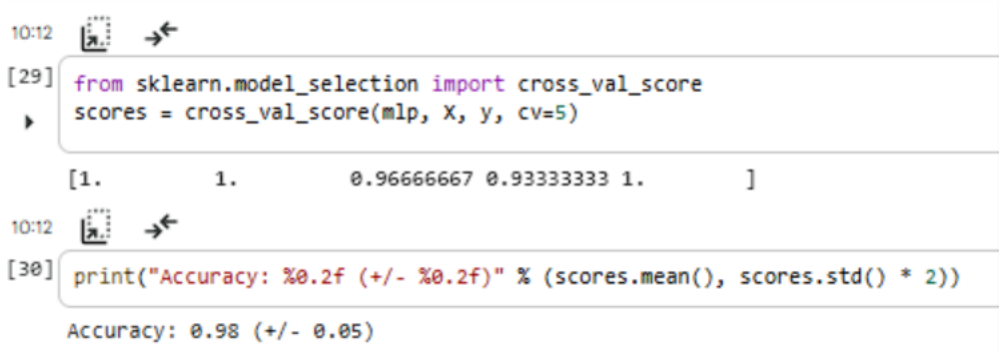


Figure 37 - cross validation

Using a (10, 5) architecture, I trained an MLP classifier that achieved 100% accuracy, confirmed by the confusion matrix. Cross-validation showed a mean score of 98% ± 5%, indicating good generalisation across splits. This architecture performed best in my experiments.

Week 9

Task 1

```
"""
    You can download the CIFAR-10 dataset from here:

    https://drive.google.com/drive/folders/17WRyo0Gduv2YppY8gIQhbUBe-LdLhZYJ?usp=sharing

    Then put all files in a folder on your Google Drive or disk, and load it to perform preprocessing to prepare
    it for the classifier.
"""

X_train, y_train, X_test, y_test = load_CIFAR10('/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/')
```

```
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/data_batch_1
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/data_batch_2
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/data_batch_3
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/data_batch_4
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/data_batch_5
/content/gdrive/My Drive/Colab Notebooks/UWL/ML_L6/CIFAR10/test_batch
```

Figure 38 - loading CIFAR-10

I loaded the CIFAR-10 dataset, which reads the training and test batch files and returns the images and labels for pre-processing and training a classifier.

```
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 7, accuracy = 0.261000
k = 7, accuracy = 0.279000
k = 7, accuracy = 0.268000
k = 7, accuracy = 0.288000
k = 7, accuracy = 0.276000
k = 9, accuracy = 0.259000
k = 9, accuracy = 0.283000
k = 9, accuracy = 0.270000
k = 9, accuracy = 0.285000
k = 9, accuracy = 0.285000
k = 11, accuracy = 0.265000
k = 11, accuracy = 0.296000
k = 11, accuracy = 0.277000
k = 11, accuracy = 0.279000
k = 11, accuracy = 0.270000
k = 13, accuracy = 0.262000
k = 13, accuracy = 0.291000
k = 13, accuracy = 0.273000
k = 13, accuracy = 0.274000
k = 13, accuracy = 0.273000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
```

Figure 39 - printing cross validation

In this screenshot, I printed the cross-validation accuracy results for each value in my K-NN classifier. For every k from 1 to 15, I displayed the accuracy across five different folds.

K = 5

```
-#best_k = # your code here
-# load the original data...
+best_k = 5
+
+# Retrain the classifier using half the training data
+# We will use the first half of the already subsampled X_train and y_train
+half_X_train = X_train[:num_training // 2]
+half_y_train = y_train[:num_training // 2]

-#classifier = KNearestNeighbor()
-#classifier.train(# your code here)
-#y_test_pred = classifier.predict(# your code here)
+classifier = KNearestNeighbor()
+classifier.train(half_X_train, half_y_train)
+y_test_pred = classifier.predict(X_test, k=best_k, num_loops=0)
```

Figure 40 - K=5

I updated my code to retrain the k-NN classifier with half the training data and set best_k = 5.

```
-#num_correct = # your code here
-#accuracy = # your code here
-#print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
+num_correct = np.sum(y_test_pred == y_test)
+accuracy = float(num_correct) / num_test
+print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 119 / 500 correct => accuracy: 0.238000
```

Figure 41 - K=5 accuracy

K=5 got 119/500 correct and an accuracy of 0.238000

K = 13

```
-#best_k = # your code here
-# load the original data...
+best_k = 13
+
+# Retrain the classifier using half the training data
+# We will use the first half of the already subsampled X_train and y_train
+half_X_train = X_train[:num_training // 2]
+half_y_train = y_train[:num_training // 2]

-#classifier = KNearestNeighbor()
-#classifier.train(# your code here)
-#y_test_pred = classifier.predict(# your code here)
+classifier = KNearestNeighbor()
+classifier.train(half_X_train, half_y_train)
+y_test_pred = classifier.predict(X_test, k=best_k, num_loops=0)
```

Figure 42 - k=13

I revised my code to retrain the k-NN classifier using half of the training data and set k to 13.

```
-#num_correct = # your code here
-#accuracy = # your code here
-#print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
+num_correct = np.sum(y_test_pred == y_test)
+accuracy = float(num_correct) / num_test
+print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 127 / 500 correct => accuracy: 0.254000
```

Figure 43 - k=13 accuracy

K = 13 got 127/500 correct and an accuracy of 0.254000

K = 15

```
-#best_k = # your code here
-# load the original data...
+best_k = 15
+
+# Retrain the classifier using half the training data
+# We will use the first half of the already subsampled X_train and y_train
+half_X_train = X_train[:num_training // 2]
+half_y_train = y_train[:num_training // 2]

-#classifier = KNearestNeighbor()
-#classifier.train(# your code here)
-#y_test_pred = classifier.predict(# your code here)
+classifier = KNearestNeighbor()
+classifier.train(half_X_train, half_y_train)
+y_test_pred = classifier.predict(X_test, k=best_k, num_loops=0)
```

Figure 44 - k=15

I finally changed my code to retrain the k-NN training data and set k to 15.

```
-#num_correct = # your code here
-#accuracy = # your code here
-#print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
+num_correct = np.sum(y_test_pred == y_test)
+accuracy = float(num_correct) / num_test
+print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 123 / 500 correct => accuracy: 0.246000
```

Figure 45 - k=15 accuracy

K = 13 got 127/500 correct and an accuracy of 0.246000

Evaluation

I selected k = 13 as the optimal value for my k-NN classifier because it consistently delivered high, stable accuracy across all five cross-validation folds. This choice balanced bias and variance effectively, minimising noise sensitivity while maintaining precise decision boundaries.

Week 10

Task 1

```
import numpy as np
# Source image region (3×3)
source = np.array([
    [3, 6, 1],
    [2, 6, 2],
    [2, 4, 1]
])
# Convolution filter (3×3)
kernal = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
])
# Element-wise multiply and sum
result = np.sum(source * kernal)

print("pixel value:", result)
```

```
pixel value: -3
```

Figure 46 - breakdown of convolution

Convolution Calculation Breakdown

- I multiplied the top row values:
  3 x (-1) = -3
  6 x 0 = 0
  1 x 1 = 1
  Total: -2
- I multiplied the middle row values:
  2 x (-2) = -4
  6 x 0 = 0
  2 x 2 = 4
  Total: 0
- Multiplied the bottom row values:
  2 x (-1) = -2
  4 x 0 =0
  1 x 1 = 1
  Total: -1
  Sum of all three rows: -3

Task 2

Kernal 3, 3

10 Epoch

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

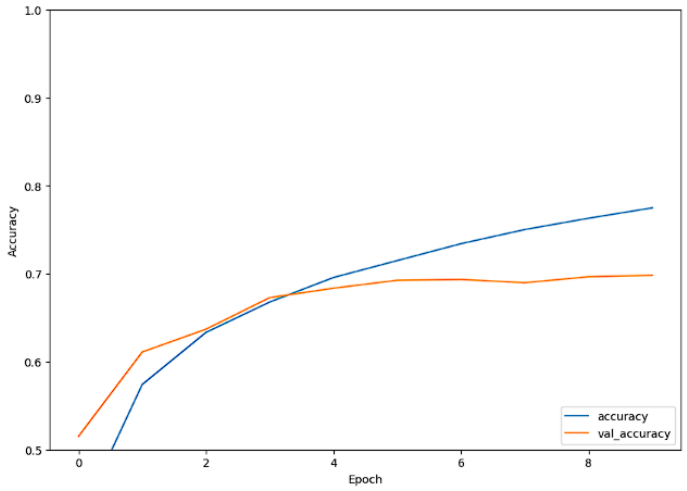
```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.3207 - loss: 1.8318 - val_accuracy: 0.5151 - val_loss: 1.3467
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5547 - loss: 1.2557 - val_accuracy: 0.6110 - val_loss: 1.1115
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6232 - loss: 1.0706 - val_accuracy: 0.6369 - val_loss: 1.0444
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6621 - loss: 0.9616 - val_accuracy: 0.6728 - val_loss: 0.9488
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6939 - loss: 0.8696 - val_accuracy: 0.6835 - val_loss: 0.9055
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7182 - loss: 0.8072 - val_accuracy: 0.6925 - val_loss: 0.8856
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.7361 - loss: 0.7540 - val_accuracy: 0.6934 - val_loss: 0.8905
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7540 - loss: 0.7115 - val_accuracy: 0.6897 - val_loss: 0.9095
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7663 - loss: 0.6719 - val_accuracy: 0.6965 - val_loss: 0.9001
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7798 - loss: 0.6321 - val_accuracy: 0.6981 - val_loss: 0.8950
```

Figure 47 - printing 10 epochs

I just ran 10 epochs of training on the model.

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7a9f65ff30e0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7045 - loss: 0.8788
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6980999708175659
```

Figure 48 - 10 epoch accuracy

After running the code, I evaluated the model on the test set, achieving about 70% accuracy, indicating that the model generalised well to unseen data.

20 Epoch

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=20,
                    validation_data=(test_images, test_labels))
```

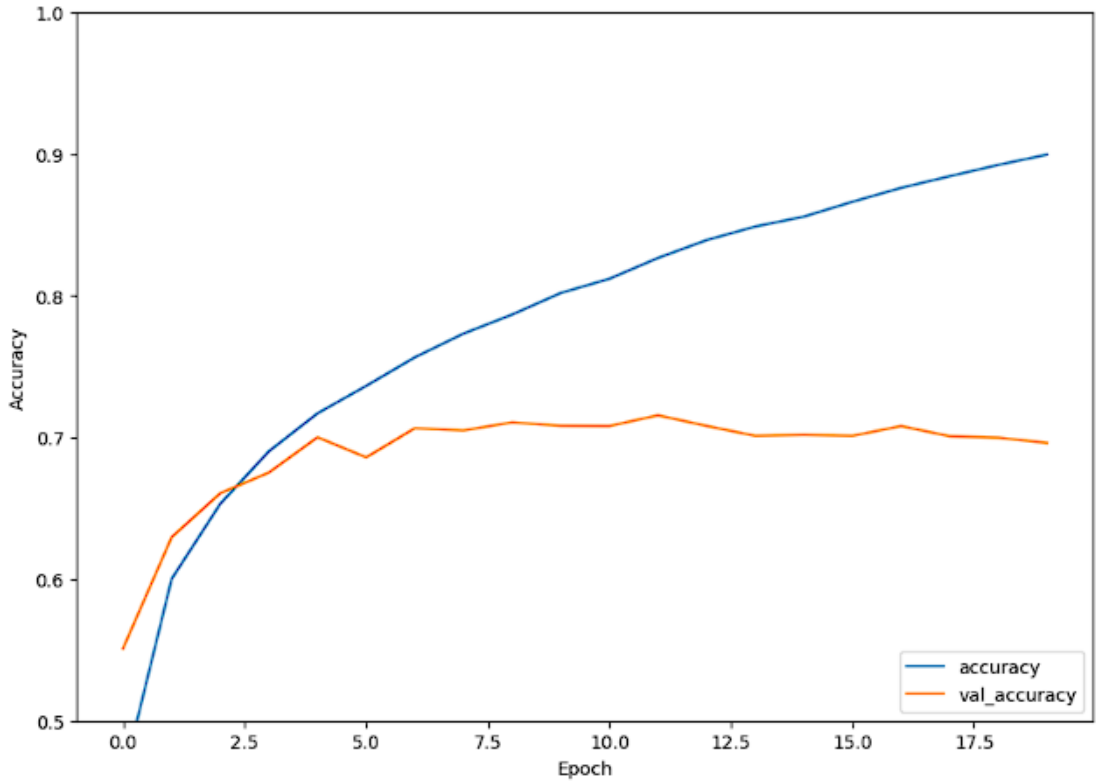
```
Epoch 1/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.3681 - loss: 1.7184 - val_accuracy: 0.5511 - val_loss: 1.2437
Epoch 2/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5890 - loss: 1.1664 - val_accuracy: 0.6296 - val_loss: 1.0579
Epoch 3/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6460 - loss: 0.9980 - val_accuracy: 0.6606 - val_loss: 0.9634
Epoch 4/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6883 - loss: 0.8788 - val_accuracy: 0.6752 - val_loss: 0.9373
Epoch 5/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7158 - loss: 0.8119 - val_accuracy: 0.7001 - val_loss: 0.8746
Epoch 6/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7377 - loss: 0.7424 - val_accuracy: 0.6859 - val_loss: 0.9202
Epoch 7/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7587 - loss: 0.6901 - val_accuracy: 0.7065 - val_loss: 0.8724
Epoch 8/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7802 - loss: 0.6306 - val_accuracy: 0.7049 - val_loss: 0.8662
Epoch 9/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7914 - loss: 0.5933 - val_accuracy: 0.7106 - val_loss: 0.8683
Epoch 10/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8065 - loss: 0.5474 - val_accuracy: 0.7082 - val_loss: 0.8820
Epoch 11/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.8175 - loss: 0.5133 - val_accuracy: 0.7080 - val_loss: 0.9032
Epoch 12/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8351 - loss: 0.4640 - val_accuracy: 0.7156 - val_loss: 0.8985
Epoch 13/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8470 - loss: 0.4308 - val_accuracy: 0.7081 - val_loss: 0.9444
Epoch 14/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8561 - loss: 0.4075 - val_accuracy: 0.7011 - val_loss: 1.0109
Epoch 15/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8665 - loss: 0.3703 - val_accuracy: 0.7018 - val_loss: 1.0259
Epoch 16/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8736 - loss: 0.3510 - val_accuracy: 0.7011 - val_loss: 1.1003
Epoch 17/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8799 - loss: 0.3311 - val_accuracy: 0.7080 - val_loss: 1.0794
Epoch 18/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8952 - loss: 0.2950 - val_accuracy: 0.7008 - val_loss: 1.1717
Epoch 19/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9002 - loss: 0.2780 - val_accuracy: 0.6998 - val_loss: 1.2025
Epoch 20/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 4ms/step - accuracy: 0.9110 - loss: 0.2487 - val_accuracy: 0.6961 - val_loss: 1.2721
```

Figure 49 - printing 20 epochs

This is the result of 20 epochs.

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7a9fd0e8b4a0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7008 - loss: 1.2668
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6960999965667725
```

Figure 50 - 20 epoch accuracy

The training accuracy increased, but the validation accuracy plateaued at around 70%, suggesting slight overfitting. The final test accuracy was 0.6961 with a test loss of 1.2668, suggesting the model generalises reasonably but could benefit from tuning or regularisation.

50 epochs

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=50,
                    validation_data=(test_images, test_labels))
```

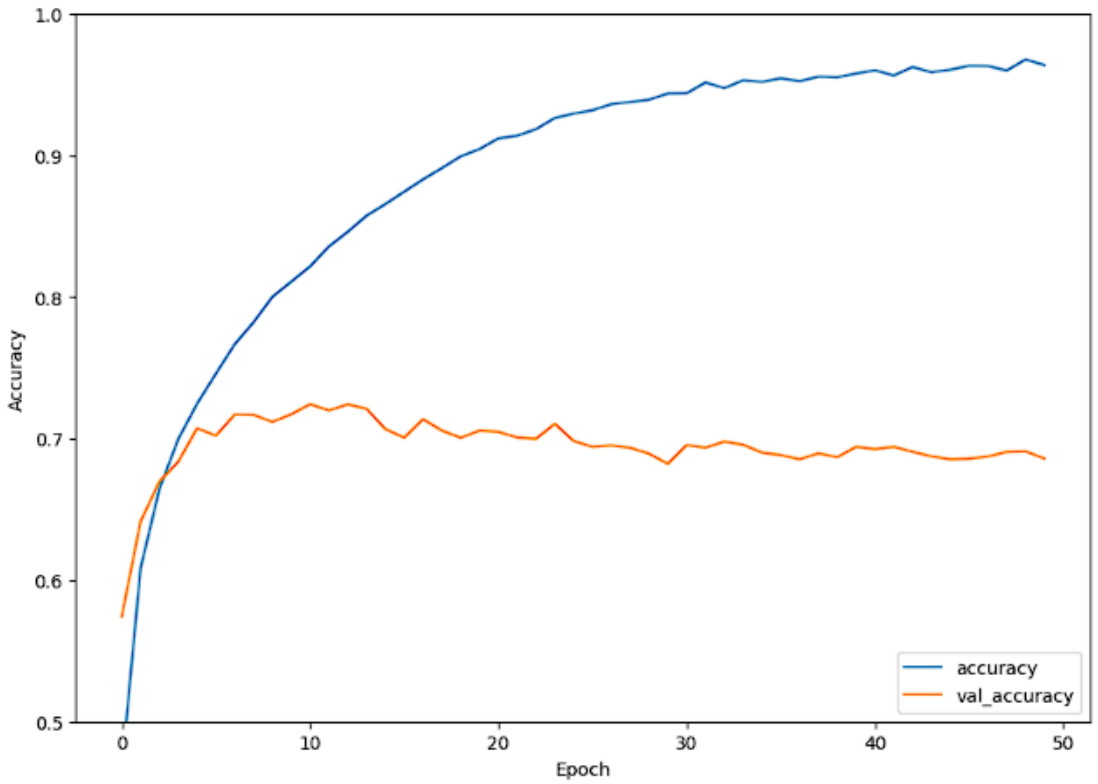
```
Epoch 1/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.3721 - loss: 1.7114 - val_accuracy: 0.5743 - val_loss: 1.1953
Epoch 2/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5953 - loss: 1.1427 - val_accuracy: 0.6414 - val_loss: 1.0192
Epoch 3/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6597 - loss: 0.9694 - val_accuracy: 0.6696 - val_loss: 0.9465
Epoch 4/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6972 - loss: 0.8607 - val_accuracy: 0.6836 - val_loss: 0.9126
Epoch 5/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7280 - loss: 0.7836 - val_accuracy: 0.7072 - val_loss: 0.8516
Epoch 6/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7487 - loss: 0.7167 - val_accuracy: 0.7020 - val_loss: 0.8684
Epoch 7/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7723 - loss: 0.6566 - val_accuracy: 0.7169 - val_loss: 0.8462
Epoch 8/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.7866 - loss: 0.6086 - val_accuracy: 0.7167 - val_loss: 0.8363
Epoch 9/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8101 - loss: 0.5477 - val_accuracy: 0.7117 - val_loss: 0.8878
Epoch 10/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8172 - loss: 0.5182 - val_accuracy: 0.7171 - val_loss: 0.8774
Epoch 11/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8286 - loss: 0.4808 - val_accuracy: 0.7242 - val_loss: 0.8788
Epoch 12/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8434 - loss: 0.4462 - val_accuracy: 0.7198 - val_loss: 0.9071
Epoch 13/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8520 - loss: 0.4167 - val_accuracy: 0.7241 - val_loss: 0.9446
Epoch 14/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8662 - loss: 0.3759 - val_accuracy: 0.7210 - val_loss: 0.9745
Epoch 15/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8737 - loss: 0.3467 - val_accuracy: 0.7066 - val_loss: 1.0463
Epoch 16/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8844 - loss: 0.3292 - val_accuracy: 0.7006 - val_loss: 1.1230
Epoch 17/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8900 - loss: 0.3095 - val_accuracy: 0.7136 - val_loss: 1.1454
Epoch 18/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8997 - loss: 0.2803 - val_accuracy: 0.7056 - val_loss: 1.2014
Epoch 19/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9064 - loss: 0.2612 - val_accuracy: 0.7005 - val_loss: 1.2536
Epoch 20/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9167 - loss: 0.2337 - val_accuracy: 0.7056 - val_loss: 1.2872
Epoch 21/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9194 - loss: 0.2254 - val_accuracy: 0.7047 - val_loss: 1.3258
Epoch 22/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9210 - loss: 0.2165 - val_accuracy: 0.7007 - val_loss: 1.3638
Epoch 23/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9294 - loss: 0.1917 - val_accuracy: 0.6998 - val_loss: 1.4307
Epoch 24/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9350 - loss: 0.1827 - val_accuracy: 0.7104 - val_loss: 1.4621
Epoch 25/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9387 - loss: 0.1701 - val_accuracy: 0.6982 - val_loss: 1.5852
Epoch 26/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9397 - loss: 0.1685 - val_accuracy: 0.6941 - val_loss: 1.7014
Epoch 27/50
```

Figure 51 - printing 50 epoch

This is the result of the 50 epochs.

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7a9ef8b5b0e0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6911 - loss: 2.6671
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6858000159263611
```

Figure 52 - 50 epoch accuracy

In this training run, I extended the model to 50 epochs to observe long-term learning behaviour. The training accuracy steadily rose past 90%, indicating good fitting. However, validation accuracy peaked around 72% early on and then fluctuated or declined slightly, while validation loss increased, suggesting overfitting. The final test accuracy was 0.6858 with a high-test loss of 2.6671, confirming the model was memorising rather than generalising.

Kernal 5, 5

```
model = models.Sequential()
model.add(layers.Conv2D(32, (5, 2), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (5, 5 ), activation='relu'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Figure 53 - inserting 5, 5 kernel

10 epoch

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10
                  ,
                    validation_data=(test_images, test_labels))
```

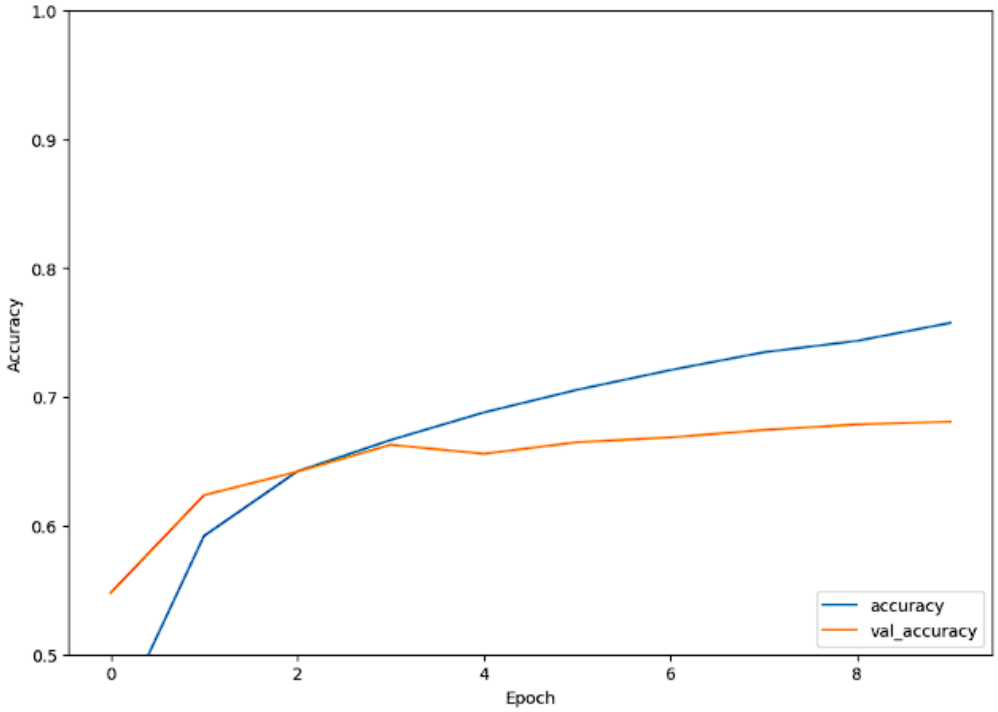
```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.3359 - loss: 1.7789 - val_accuracy: 0.5479 - val_l
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5749 - loss: 1.1896 - val_accuracy: 0.6237 - val_lo
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6358 - loss: 1.0350 - val_accuracy: 0.6419 - val_lo
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6679 - loss: 0.9490 - val_accuracy: 0.6628 - val_lo
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6888 - loss: 0.8837 - val_accuracy: 0.6558 - val_lo
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7038 - loss: 0.8406 - val_accuracy: 0.6648 - val_lo
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7283 - loss: 0.7766 - val_accuracy: 0.6685 - val_lo
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7399 - loss: 0.7489 - val_accuracy: 0.6743 - val_lo
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7496 - loss: 0.7123 - val_accuracy: 0.6786 - val_lo
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7629 - loss: 0.6819 - val_accuracy: 0.6807 - val_lo
```

Figure 54 - printing 10 epochs

This is the final visualisation of the 10 epochs in kernel 5, 5.

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x78c678799be0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6895 - loss: 0.9341
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6807000041007996
```

Figure 55 - accuracy on 10 epoch

The test accuracy reached approximately 68.9% by the final epoch.

20 epoch

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=20
                    ,
                      validation_data=(test_images, test_labels))
```

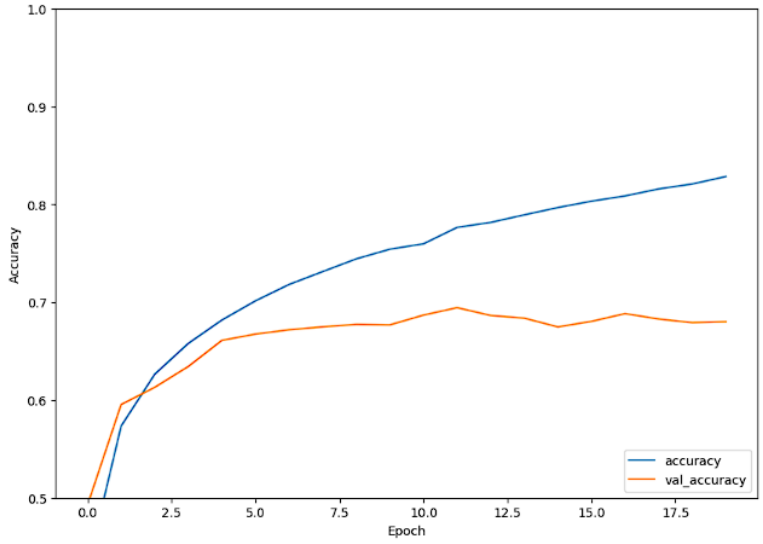
```
Epoch 1/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 5ms/step - accuracy: 0.3380 - loss: 1.7883 - val_accuracy: 0.4927 - val_loss: 1.3941
Epoch 2/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5552 - loss: 1.2484 - val_accuracy: 0.5954 - val_loss: 1.1499
Epoch 3/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6208 - loss: 1.0719 - val_accuracy: 0.6130 - val_loss: 1.1253
Epoch 4/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6599 - loss: 0.9715 - val_accuracy: 0.6343 - val_loss: 1.0376
Epoch 5/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6786 - loss: 0.9149 - val_accuracy: 0.6610 - val_loss: 0.9899
Epoch 6/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7058 - loss: 0.8368 - val_accuracy: 0.6674 - val_loss: 0.9589
Epoch 7/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7190 - loss: 0.7983 - val_accuracy: 0.6718 - val_loss: 0.9608
Epoch 8/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7375 - loss: 0.7582 - val_accuracy: 0.6748 - val_loss: 0.9392
Epoch 9/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7467 - loss: 0.7218 - val_accuracy: 0.6773 - val_loss: 0.9560
Epoch 10/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7578 - loss: 0.6913 - val_accuracy: 0.6768 - val_loss: 0.9469
Epoch 11/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7672 - loss: 0.6574 - val_accuracy: 0.6868 - val_loss: 0.9507
Epoch 12/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7840 - loss: 0.6220 - val_accuracy: 0.6944 - val_loss: 0.9425
Epoch 13/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7881 - loss: 0.5967 - val_accuracy: 0.6864 - val_loss: 0.9781
Epoch 14/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7975 - loss: 0.5794 - val_accuracy: 0.6837 - val_loss: 0.9904
Epoch 15/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8010 - loss: 0.5599 - val_accuracy: 0.6747 - val_loss: 1.0117
Epoch 16/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8113 - loss: 0.5401 - val_accuracy: 0.6804 - val_loss: 1.0015
Epoch 17/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8188 - loss: 0.5143 - val_accuracy: 0.6883 - val_loss: 1.0621
Epoch 18/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8235 - loss: 0.5030 - val_accuracy: 0.6828 - val_loss: 1.0333
Epoch 19/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8285 - loss: 0.4809 - val_accuracy: 0.6791 - val_loss: 1.0737
Epoch 20/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8342 - loss: 0.4646 - val_accuracy: 0.6801 - val_loss: 1.1088
```

Figure 56 - printing 20 epochs

This is the final visualisation for 20 epochs

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x78c75a0c34d0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6859 - loss: 1.0978
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6801000237464905
```

Figure 57 - accuracy for 20 epochs

The test accuracy for 20 epochs in kernel 5, 5 is approximately 68%

50 epochs

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=50
                    ,
                      validation_data=(test_images, test_labels))
```

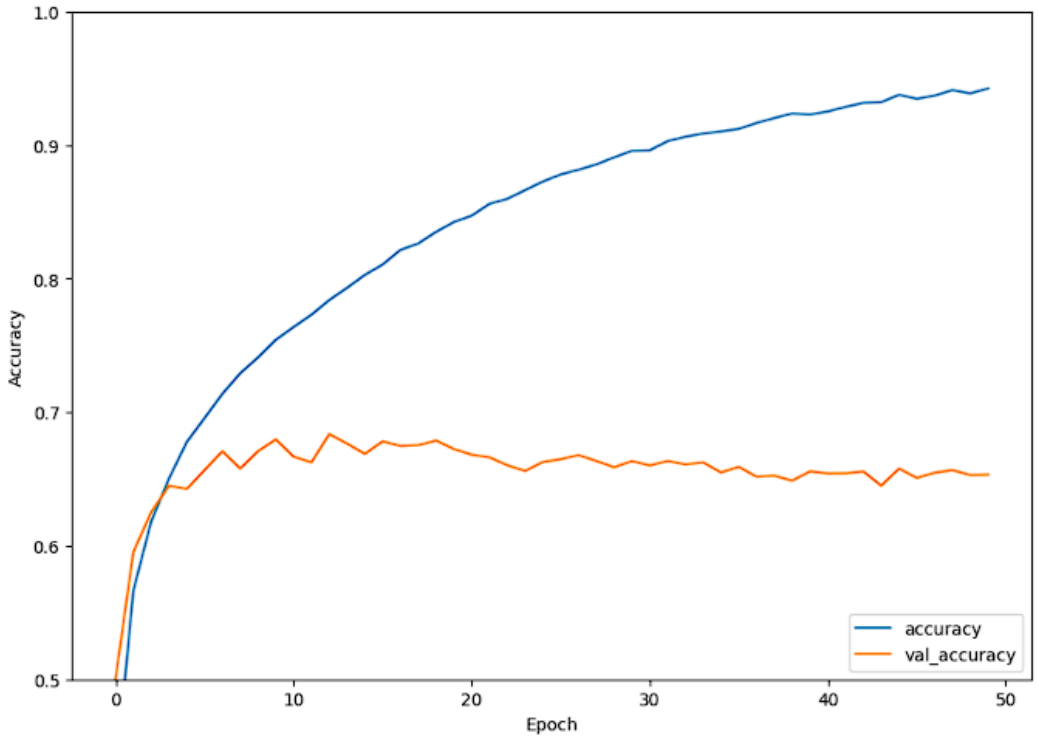
```
Epoch 1/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.3394 - loss: 1.7891 - val_accuracy: 0.5004 - val_loss: 1.3785
Epoch 2/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5508 - loss: 1.2529 - val_accuracy: 0.5954 - val_loss: 1.1274
Epoch 3/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6141 - loss: 1.0851 - val_accuracy: 0.6251 - val_loss: 1.0593
Epoch 4/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6487 - loss: 0.9947 - val_accuracy: 0.6450 - val_loss: 1.0031
Epoch 5/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6790 - loss: 0.9132 - val_accuracy: 0.6425 - val_loss: 1.0248
Epoch 6/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6996 - loss: 0.8512 - val_accuracy: 0.6567 - val_loss: 0.9848
Epoch 7/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7130 - loss: 0.8081 - val_accuracy: 0.6708 - val_loss: 0.9652
Epoch 8/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7292 - loss: 0.7639 - val_accuracy: 0.6578 - val_loss: 1.0125
Epoch 9/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7434 - loss: 0.7261 - val_accuracy: 0.6708 - val_loss: 0.9513
Epoch 10/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7632 - loss: 0.6742 - val_accuracy: 0.6796 - val_loss: 0.9684
Epoch 11/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7714 - loss: 0.6470 - val_accuracy: 0.6667 - val_loss: 1.0027
Epoch 12/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7790 - loss: 0.6258 - val_accuracy: 0.6625 - val_loss: 1.0435
Epoch 13/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7907 - loss: 0.5894 - val_accuracy: 0.6836 - val_loss: 0.9946
Epoch 14/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8017 - loss: 0.5632 - val_accuracy: 0.6765 - val_loss: 1.0371
Epoch 15/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8111 - loss: 0.5342 - val_accuracy: 0.6688 - val_loss: 1.0588
Epoch 16/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8172 - loss: 0.5147 - val_accuracy: 0.6782 - val_loss: 1.1034
Epoch 17/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8299 - loss: 0.4828 - val_accuracy: 0.6747 - val_loss: 1.0847
Epoch 18/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8372 - loss: 0.4636 - val_accuracy: 0.6753 - val_loss: 1.1095
Epoch 19/50
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8459 - loss: 0.4355 - val_accuracy: 0.6788 - val_loss: 1.1369
Epoch 20/50
```

Figure 58 - printing 50 epochs

Finally this is the findings for 50 epochs

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x78c6e00ebcb0>
```




```
test_loss, test_acc = model.evaluate(test_images,  test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6470 - loss: 2.5327
```

```
print('Test accuracy:', test_acc)
```

```
Test accuracy: 0.6532999873161316
```

Figure 59 - accuracy for 50 epochs

After training the model, the validation and test accuracy was 65.3%, which is slightly lower than the other two (10 and 20 epochs)

Evaluation

Overall, my experiments showed that using a (3,3) kernel in the CNN model led to slightly better generalisation and higher test accuracy compared to the (5,5) kernel. Both setups performed slightly, which helped the model perform better on unseen data. Increasing the number of epochs to 50 didn't improve accuracy; in fact, it caused overfitting, with training accuracy rising sharply while validation accuracy plateaued. This simply suggests that simply training longer or using larger kernals doesn't guarantee better results; instead, careful tuning and regularisation are key to improving performance.

Week 11

Task 1

y = 6.49x + 30.18:

6.49(6) + 30.18 = 69.12

6.49(10) + 30.18 = 94.08

6.49(2) + 30.18 = 43.16

6.49(4) + 30.18 = 56.14

6.49(6) + 30.18 = 69.12

6.49(7) + 30.18 = 75.61

6.49(0) + 30.18 = 30.18

6.49(1) + 30.18 = 36.67

6.49(8) + 30.18 = 82.10

6.49(5) + 30.18 = 62.63

6.49(3) + 30.18 = 49.65

LSE=165.89+36.97+164.86+61.78+62.09+268.65+51.55+18.75+4.41+13.18+7.02=

855.15

Task 2

Model A, Year 1-9

Year	Advertising	Sales		Residual	Squared Residual
1	23	651	22.60(23) + 169.2 = 689	651 – 689 = -38	$(-38)^2$=1444
2	25	762	22.60(25) + 169.2 = 734.2	762 - 734.2 = 27.8	$27.8)^2$= 772.84
3	30	856	22.60(30) + 169.2 = 847.2	856 - 847.2 = 8.8	$(8.8)^2$= 77.44
4	34	1063	22.60(34) + 169.2 = 937.6	1063 - 937.6 = 125.4	$(125.4)^2$= 15714.96
5	43	1190	22.60(43) + 169.2 = 1139	1190 – 1139 = 51	$(51)^2$= 2601
6	44	1298	22.60(44) + 169.2 = 1161.6	1298 - 1161.6 = 136.4	$(136.4)^2$= 18605
7	52	1421	22.60(52) + 169.2 = 1340.4	1421 - 1340.4 = 80.6	$(80.6)^2$= 6496.36
8	57	1440	22.60(57) + 169.2 = 1462.4	1440 - 1462.4 = -22.4	$(-22.4)^2$= 501.76
9	58	1518	22.60(58) + 169.2 = 1485	1518 – 1485 = 33	$(33)^2$= 1089

LSE = sum of Squared Residential = 47302.36

Model B, Year 1–9

Equation:

Sales = 23.42(Advertising) + 167.7

Year	Advertising	Sales		Residual	Squared Residual
1	23	651	23.42(23) + 167.7 = 706.36	651 − 706.36 = -55.36	$(-55.36)^2 = 3064.73$
2	25	762	23.42(25) + 167.7 = 753.20	762 − 753.20 = 8.80	$(8.80)^2 = 77.44$
3	30	856	23.42(30) + 167.7 = 870.30	856 − 870.30 = -14.30	$(-14.30)^2 = 204.49$
4	34	1063	23.42(34) + 167.7 = 963.98	1063 − 963.98 = 99.02	$(99.02)^2 = 9804.96$
5	43	1190	23.42(43) + 167.7 = 1174.76	1190 − 1174.76 = 15.24	$(15.24)^2 = 232.26$
6	44	1298	23.42(44) + 167.7 = 1198.18	1298 − 1198.18 = 99.82	$(99.82)^2 = 9964.03$
7	52	1421	23.42(52) + 167.7 = 1385.54	1421 − 1385.54 = 35.46	$(35.46)^2 = 1257.41$
8	57	1440	23.42(57) + 167.7 = 1502.64	1440 − 1502.64 = -62.64	$(-62.64)^2 = 3923.77$
9	58	1518	23.42(58) + 167.7 = 1526.06	1518 − 1526.06 = -8.06	$(-8.06)^2 = 64.96$

LSE = sum of Squared Residential = 27,735.6

Year	Advertising	Sales	LSE	Residual	Squared Residual
1	23	651	23.10(23) + 168.1 = 699.40	651 − 699.40 = -48.40	$(-48.40)^2 = 2344.56$
2	25	762	23.10(25) + 168.1 = 745.60	762 − 745.60 = 16.40	$(16.40)^2 = 268.96$
3	30	856	23.10(30) + 168.1 = 861.10	856 − 861.10 = -5.10	$(-5.10)^2 = 26.01$
4	34	1063	23.10(34) + 168.1 = 953.50	1063 − 953.50 = 109.50	$(109.50)^2 = 11990.25$
5	43	1190	23.10(43) + 168.1 = 1161.40	1190 − 1161.40 = 28.60	$(28.60)^2 = 817.96$
6	44	1298	23.10(44) + 168.1 = 1184.50	1298 − 1184.50 = 113.50	$(113.50)^2 = 12882.25$
7	52	1421	23.10(52) + 168.1 = 1369.30	1421 − 1369.30 = 51.70	$(51.70)^2 = 2672.89$
8	57	1440	23.10(57) + 168.1 = 1484.80	1440 − 1484.80 = -44.80	$(-44.80)^2 = 2007.04$
9	58	1518	23.10(58) + 168.1 = 1507.90	1518 − 1507.90 = 10.10	$(10.10)^2 = 102.01$

LSE = sum of Squared Residential = 33330.14

We can see that model B has the lowest Least Squares Error.

Task 3

```
print('Line equation: y = mx + b')
print('Optimized m:', linearRegressor.coef_)
print('Optimized b:', linearRegressor.intercept_)
```

```
Line equation: y = mx + b
Optimized m: [[1.32243102]]
Optimized b: [7.99102098]
```

Figure 60 - training linear regression

After training my linear regression model with the advertising dataset, the derived equation was Sales = 1.3224 × Marketing Spend + 7.991. This indicates that sales tend to rise by roughly £1.32 for each additional pound spent on marketing. The model forecasts sales of £7.99 when no marketing is used and about £9,917 for a £7,500 marketing budget, demonstrating a clear positive correlation between marketing expenditure and sales.

```
x_new_0 = np.array([0]).reshape(-1, 1)
x_new_7500 = np.array([7500]).reshape(-1, 1)

yPrediction_0 = linearRegressor.predict(x_new_0)
yPrediction_7500 = linearRegressor.predict(x_new_7500)

print('Predicted sale for £0 marketing spend:', yPrediction_0)
print('Predicted sale for £7500 marketing spend:', yPrediction_7500)
```

```
Predicted sale for £0 marketing spend: [[7.99102098]]
Predicted sale for £7500 marketing spend: [[9926.22369165]]
```

Figure 61 - predicted sales

I used my trained regression model to forecast sales based on marketing expenditure. Inputting £0, it predicted sales of £7.99, aligning with the equation's intercept. When I tested with £7,500, the model forecasted sales of £9,926.22. These outcomes verify the model's functionality and demonstrate a strong positive link between marketing spend and expected sales.

Task 4

```
m = len(y)
errors = yPrediction - y
J = (1/(2*m)) * np.sum(errors**2)
print("Cost function J for the fitted line:", J)
```

```
Cost function J for the fitted line: 2233.2548102901433
```

Figure 62 - function(J)

In Task 4, I computed the cost function (J) to evaluate how well my linear regression model fits the data. After training and generating predictions, I applied the formula ($J = (1/2m) \sum (\hat{y} - y)^2$), where (m) is the number of data points, ($\hat{y}$) is the predicted value, and (y) is the actual value. I implemented this in Python with NumPy and obtained a result of about 2233.25. This value indicates the average squared error of the predictions and shows that the model is fairly accurate for my dataset.

Evaluation

Across all four tasks, I used linear regression techniques on real-world datasets and confidently interpreted the results. In Task 1, I calculated the Least Squares Error to assess how accurately the regression model predicted grades based on hours spent writing a logbook. Task 2 involved comparing three different regression models using advertising spend to predict sales, and I identified the best fit by calculating and comparing their total errors. For Task 3, I trained a regression model in Python to predict sales from marketing spend, then used it to estimate outcomes for budgets of £0 and £7,500, confirming a positive correlation. Finally, in Task 4, I computed the cost function (J) to evaluate the model's accuracy, providing a clear numerical measure of prediction

quality. These tasks helped me understand both the theoretical basis and practical application of regression analysis.

References

GeeksforGeeks (2019). *knearest neighbor algorithm using Sklearn Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbor-algorithm-in-python/.

Python, R. (n.d.). *The k-Nearest Neighbors (kNN) Algorithm in Python – Real Python*. [online] realpython.com. Available at: https://realpython.com/knn-python/.

GeeksforGeeks (2018). *What is Unsupervised Learning?* [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/unsupervised-learning/.

IBM (2021). *What is a Neural Network?* [online] IBM. Available at: https://www.ibm.com/think/topics/neural-networks.